

LAPORAN PRAKTIKUM MODUL 4

Cookies, Session, dan Local Storage

Diajukan untuk memenuhi salah satu tugas praktikum mata kuliah Proyek 3 : Pengembangan Perangkat Lunak Berbasis Web



Disusun Oleh:

Nama : R. Neysa Rahma Velda

NIM : 251511057

Kelas : D3 - 2B

Repo Github https://github.com/Neysavelda/praktikum--proyek-modul-04_251511057_R.NeysaRahmaVelda

Jurusan Teknik Komputer dan Informatika

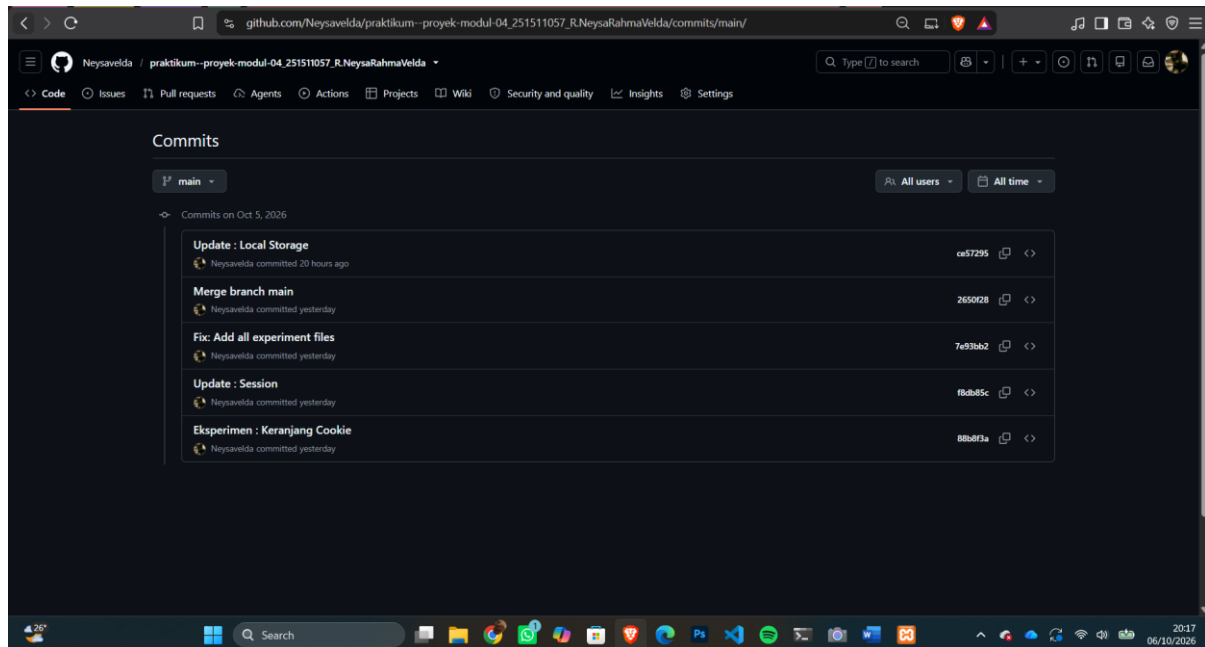
**Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung
2026**

A. Hasil Eksperimen

Bagian ini memuat hasil eksperimen dari modul Penyimpanan Data pada Web, yaitu pengantar HTTP stateless, cookies, session, Local Storage, serta perbandingan dan keamanannya. Seluruh file eksperimen sudah diunggah ke repository GitHub yang dicantumkan di atas.

A.1 Dokumentasi Repository

Eksperimen dikerjakan bertahap dan setiap tahap dicatat lewat commit pada branch main. Riwayat commit-nya ditampilkan pada gambar berikut.

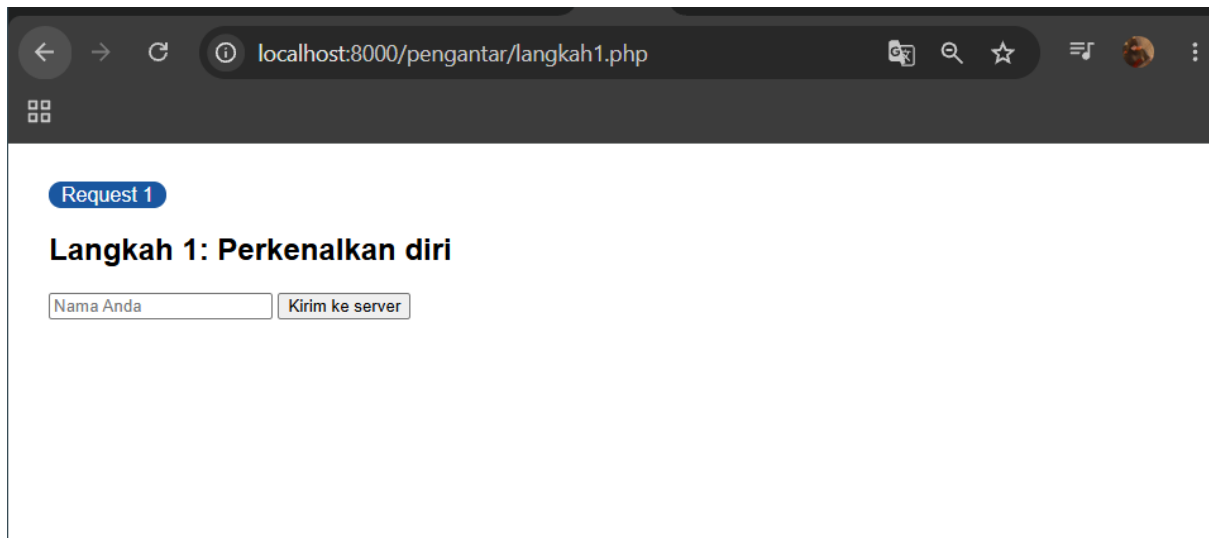


Gambar 1. Riwayat commit repository eksperimen Modul 4

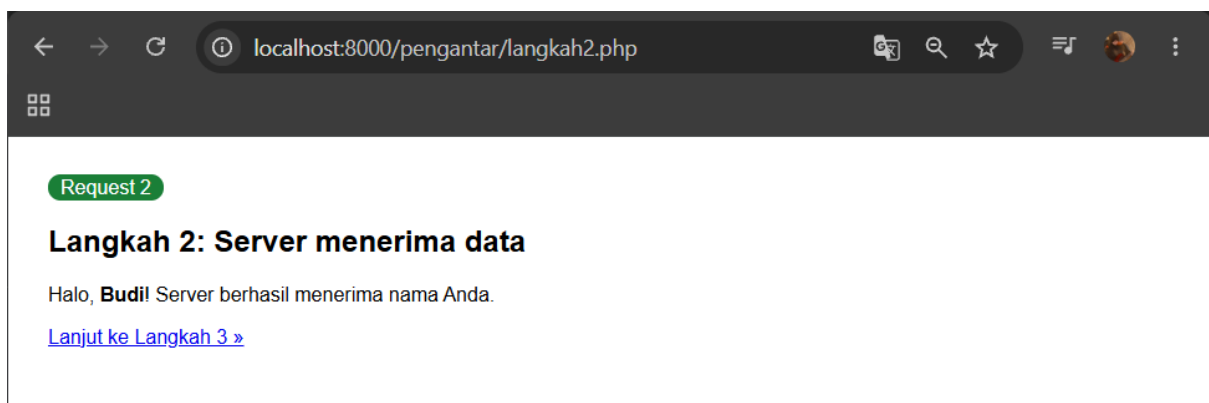
A.2 Tahap 1: HTTP Bersifat Stateless

Eksperimen memakai tiga halaman (langkah1.php, langkah2.php, dan langkah3.php) untuk membuktikan bahwa server tidak mengingat pengguna dari satu request ke request berikutnya.

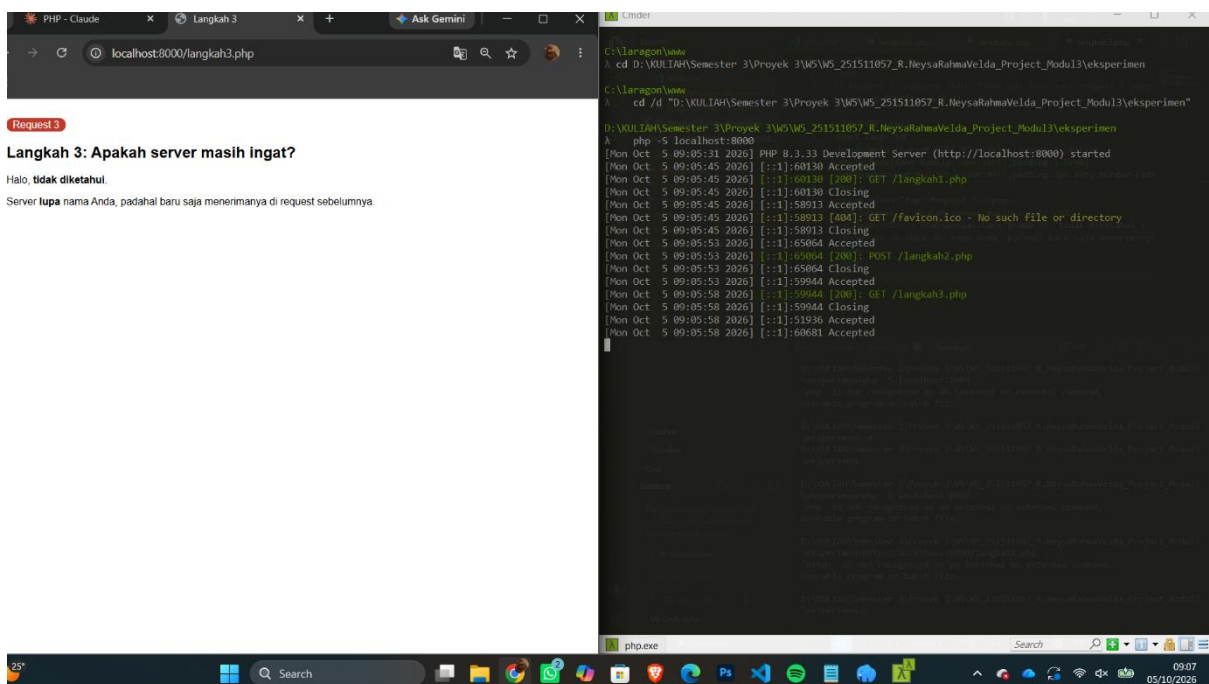
Pada Request 1, nama "Budi" diisi di form lalu dikirim. Pada Request 2, server berhasil menerima nama tersebut lewat `$_POST` dan menampilkan "Halo, Budi!". Pada Request 3, yang hanya berjarak satu klik tautan, server sudah tidak mengetahui nama itu dan menampilkan "tidak diketahui". Hal ini terjadi karena setiap request diperlakukan sebagai kejadian yang berdiri sendiri.



Gambar 2. Request 1: pengguna mengisi nama



Gambar 3. Request 2: server menerima data



Gambar 4. Request 3: server lupa nama pengguna

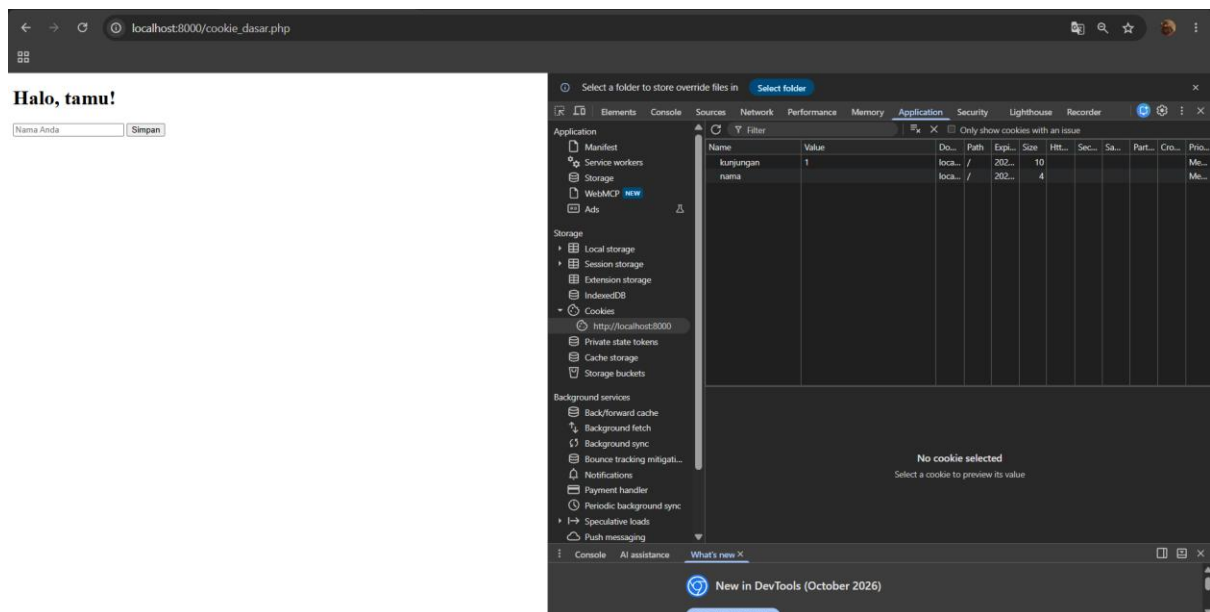
Pertanyaan diskusi: bagaimana agar langkah3.php tetap menampilkan "Halo, Budi"?

Nama harus disimpan di suatu tempat yang dibawa ke request berikutnya, misalnya session, cookie, atau Local Storage. Contohnya dengan menambahkan `$_SESSION['nama'] = $nama;` di langkah2.php, lalu membacanya kembali di langkah3.php.

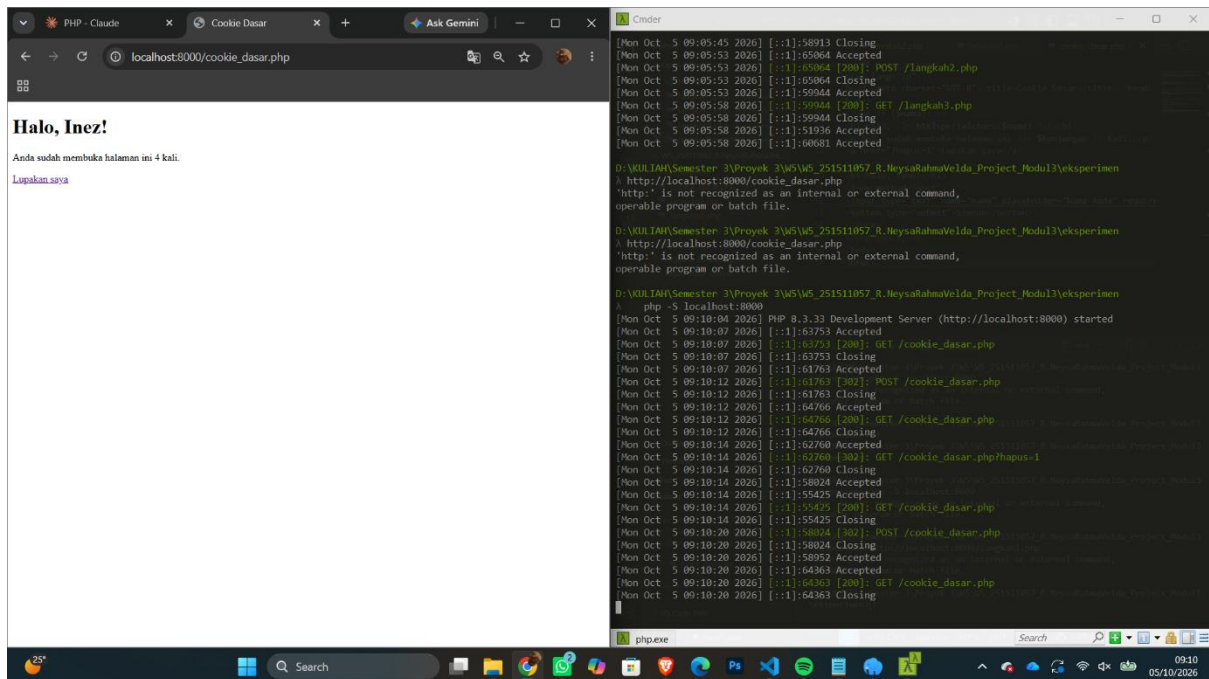
A.3 Tahap 2: Cookies (PHP)

Eksperimen 1: cookie_dasar.php

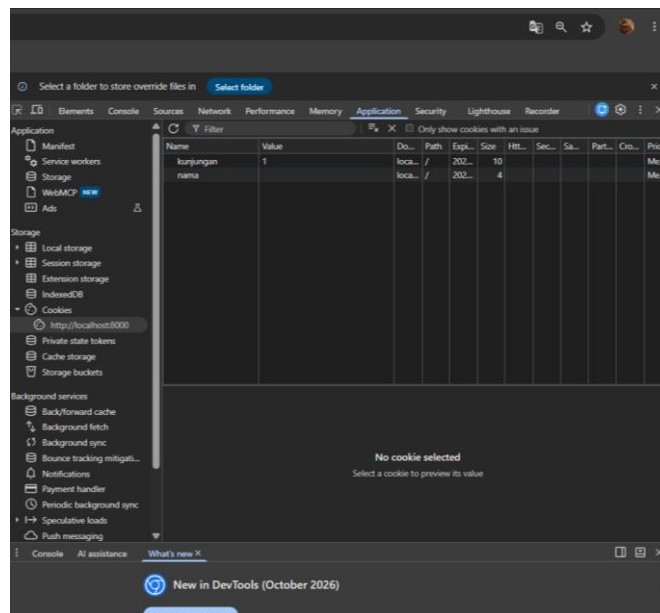
Program menyimpan nama pengguna dan jumlah kunjungan dalam cookie selama 7 hari. Pada kunjungan pertama belum ada cookie nama sehingga form input tampil. Setelah nama "Budi" disimpan dan halaman dialihkan (redirect), halaman menyapa dari isi cookie dan angka kunjungan menjadi 2. Tombol "Lupakan saya" menghapus cookie nama dengan mengatur masa berlakunya ke waktu yang sudah lewat, sehingga form tampil kembali.



Gambar 5. Kunjungan pertama: form input tampil



Gambar 6. Setelah nama disimpan: kunjungan = 4



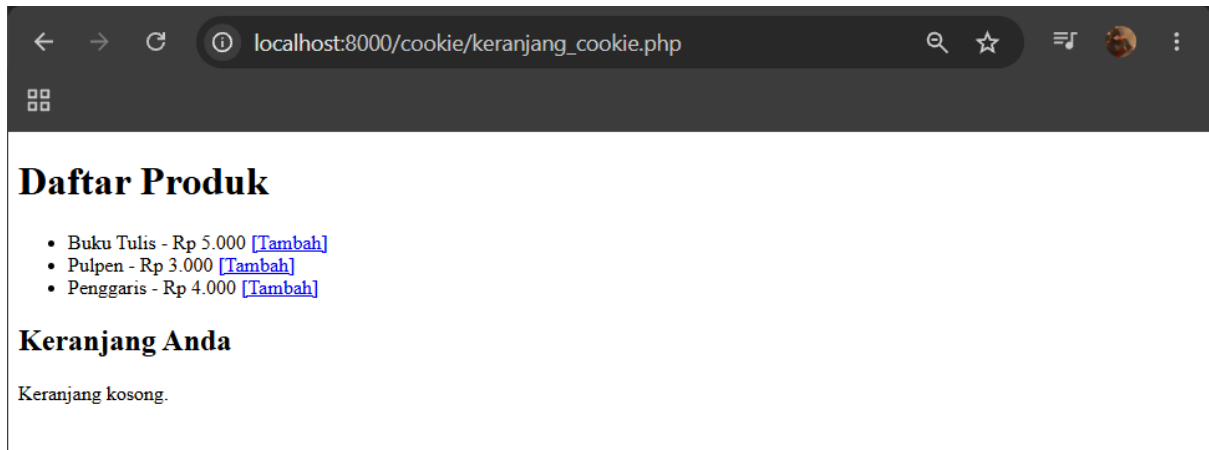
Gambar 7. Isi cookie nama dan kunjungan di DevTools

Pertanyaan: mengapa penghitung diletakkan di bawah blok exit?

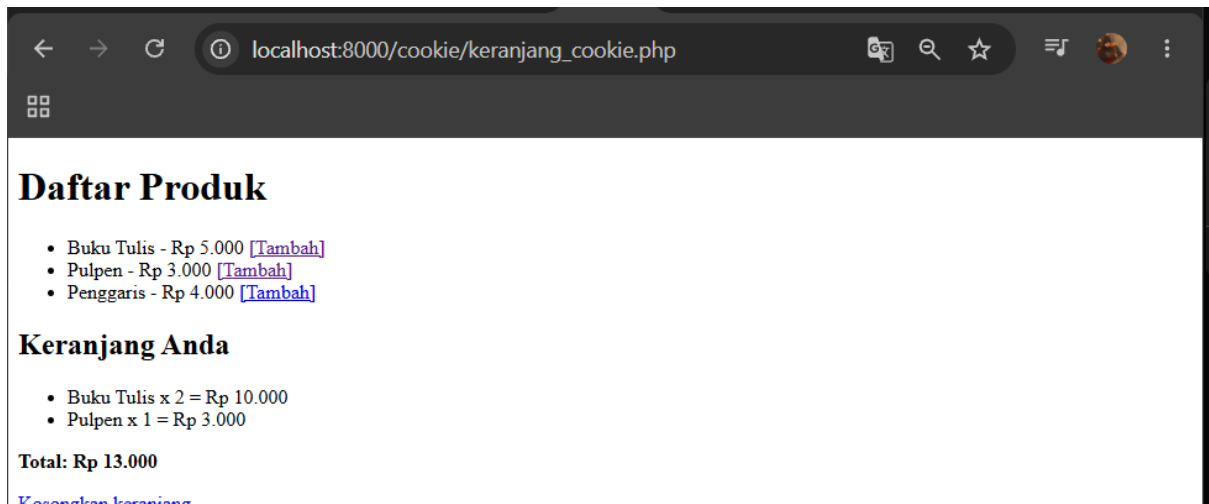
Setiap aksi form atau tautan menghasilkan dua request: request pertama berakhir di header() dan exit, lalu browser membuat request kedua akibat redirect. Jika penghitung berada di atas blok exit, kedua request ikut dihitung sehingga angka naik 2 saat menekan Simpan atau Lupakan saya. Jika penghitung berada di bawah semua exit, hanya request yang benar-benar menampilkan halaman yang dihitung, sehingga angka hanya naik 1. Pelajarannya, urutan kode di PHP menentukan hasil karena kode sebelum exit tetap dijalankan pada request yang berujung redirect.

Eksperimen 2: *keranjang_cookie.php*

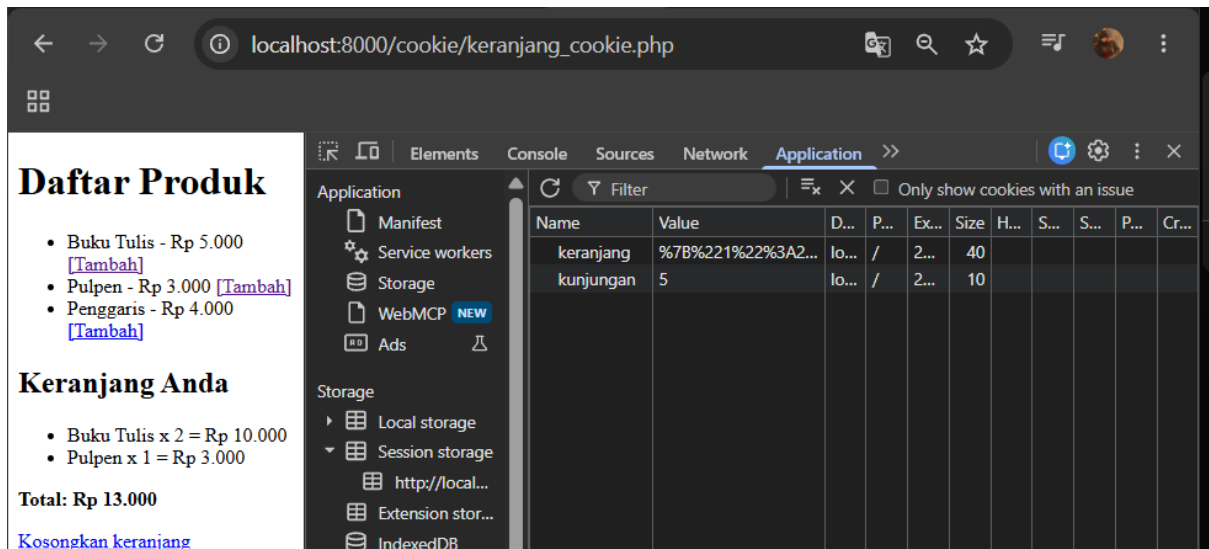
Keranjang belanja disimpan dalam cookie bernama *keranjang*. Karena cookie hanya dapat menyimpan teks, array *keranjang* diubah menjadi string JSON dengan kunci berupa id produk dan nilai berupa jumlah. Setelah Buku Tulis ditambah 2 kali dan Pulpen 1 kali, total harga tampil Rp 13.000.



Gambar 10. Keranjang kosong



Gambar 11. Keranjang terisi



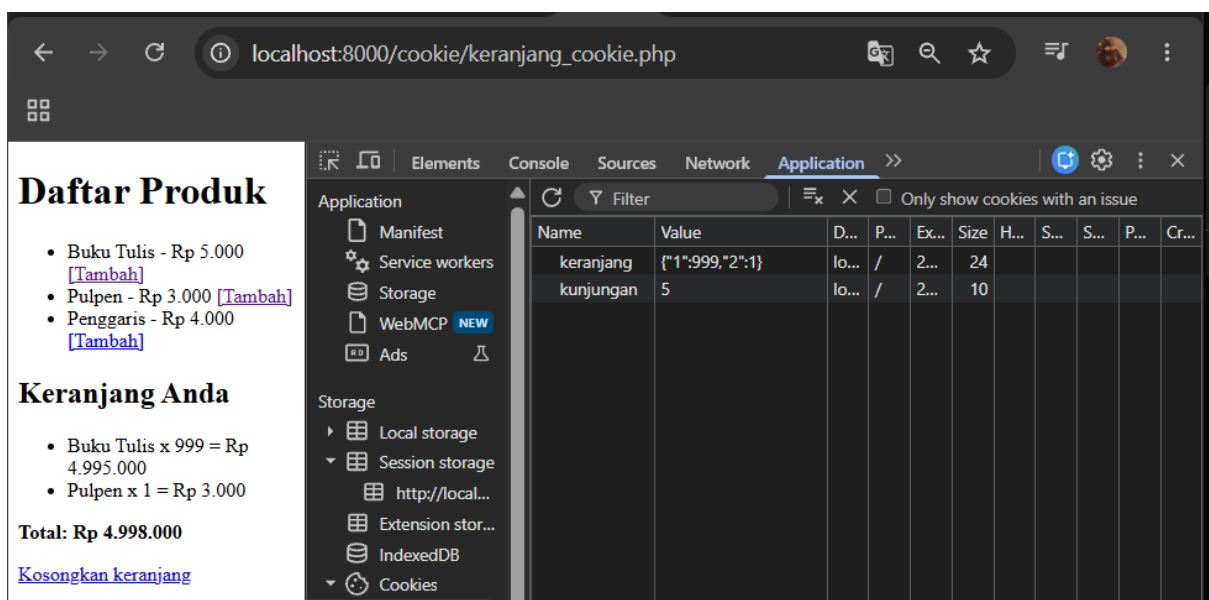
Gambar 12. Cookie keranjang di DevTools

Percobaan keamanan: mengubah cookie secara manual

Nilai cookie keranjang diubah lewat DevTools sehingga jumlah Buku Tulis menjadi 999. Setelah halaman di-refresh, server memercayai nilai tersebut dan total berubah menjadi Rp 4.998.000 (999 x Rp 5.000 ditambah Rp 3.000 untuk Pulpen). Ini membuktikan bahwa data di cookie sepenuhnya berada di tangan pengguna dan dapat dimanipulasi dengan mudah.

Name	Value	D...	P...	Ex...	Size	H...	S...	S...	P...	Cr...
keranjang	{"1":999,"2":1}	lo...	/	2...	24					
kunjungan	7	lo...	/	2...	10					
PHPSESSID	1crql13h1v5rdjp90e...	lo...	/	S...	35					

Gambar 13. Nilai cookie diubah menjadi 999



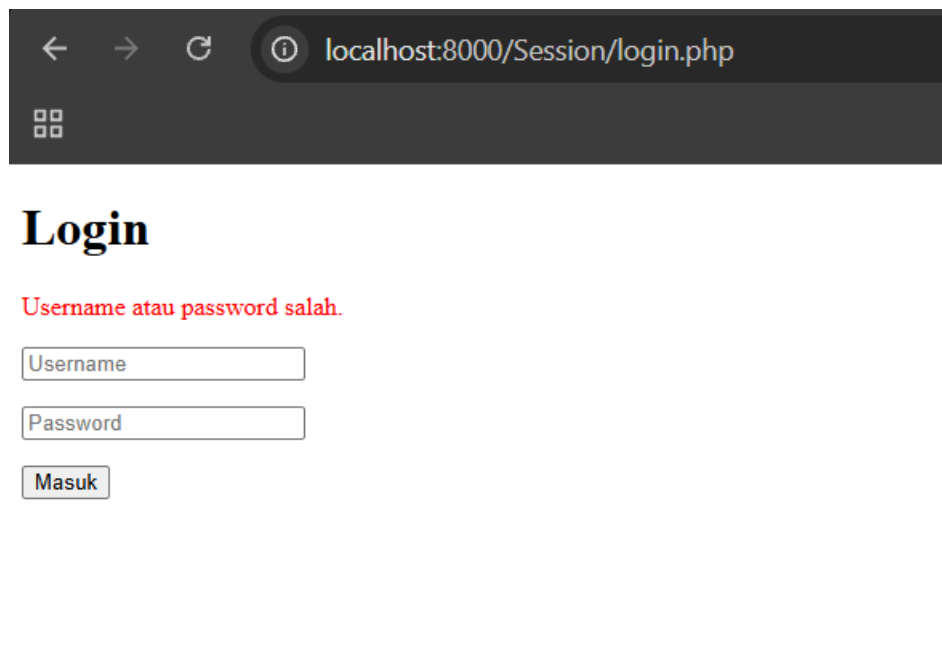
Gambar 14. Total berubah setelah refresh

Diskusi: mengapa keranjang belanja lebih baik memakai session?

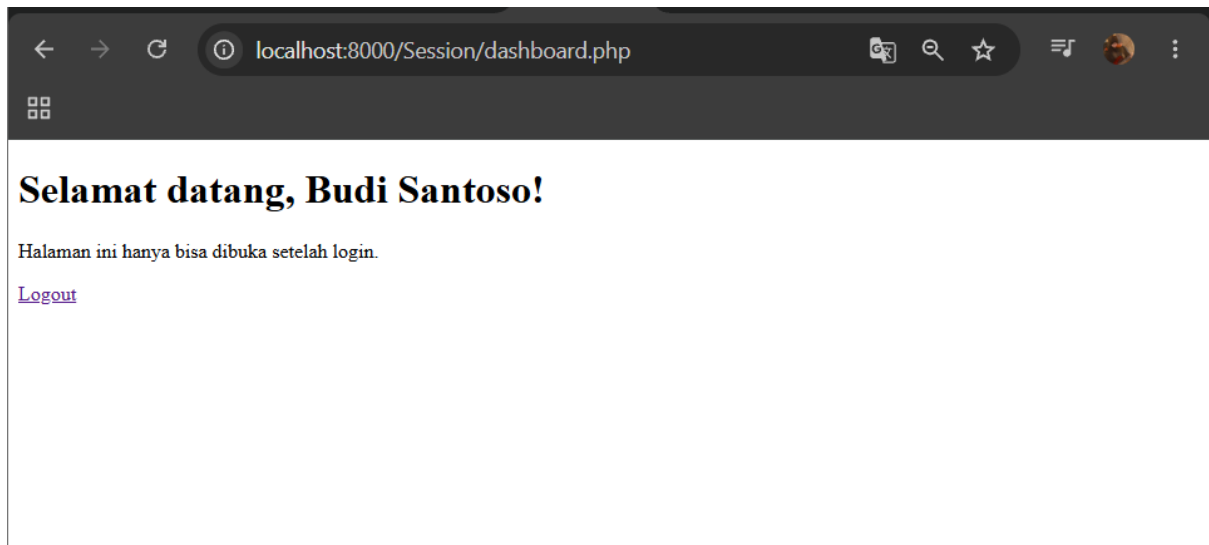
Cookie tersimpan di browser sehingga pengguna dapat melihat dan mengubah isinya, ikut terkirim pada setiap request sehingga menambah lalu lintas data, dan kapasitasnya hanya sekitar 4 KB. Pada session, data keranjang berada di server dan browser hanya menyimpan ID sesi, sehingga pengguna tidak dapat mengubah jumlah atau data lainnya secara langsung.

A.4 Tahap 3: Session (PHP)

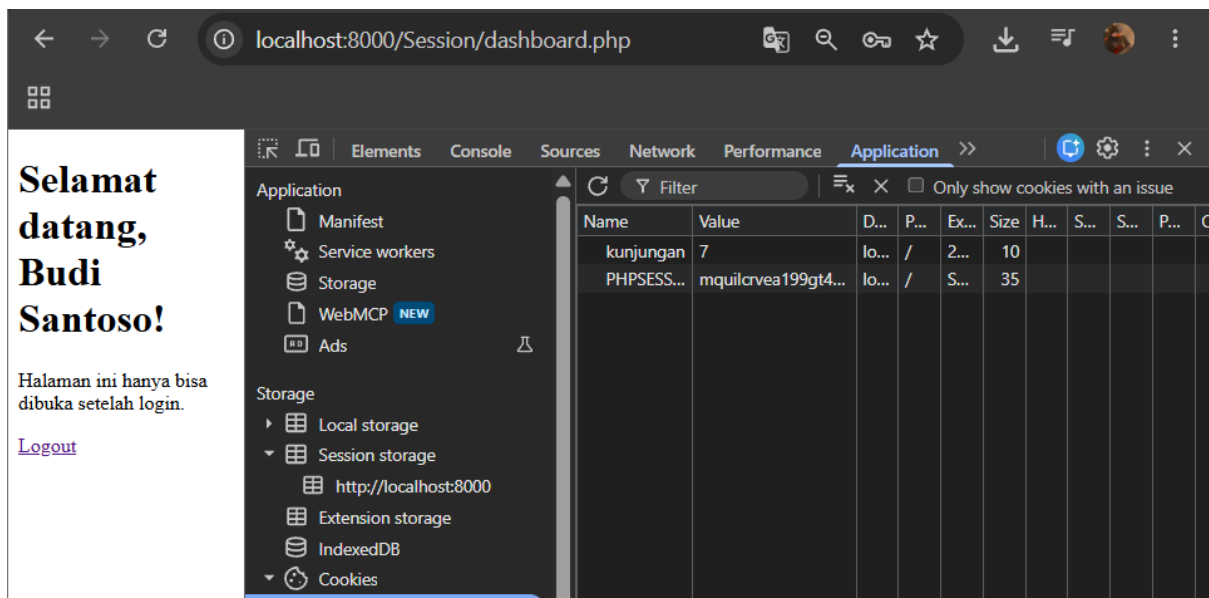
Eksperimen ini membangun sistem login sederhana dengan session. Password pengguna disimpan dalam bentuk hash dan diperiksa dengan `password_verify()`. Setelah login berhasil, `session_regenerate_id(true)` dipanggil untuk mencegah session fixation, yaitu serangan yang memaksa pengguna memakai ID sesi yang sudah diketahui penyerang.



Gambar 15. Login gagal

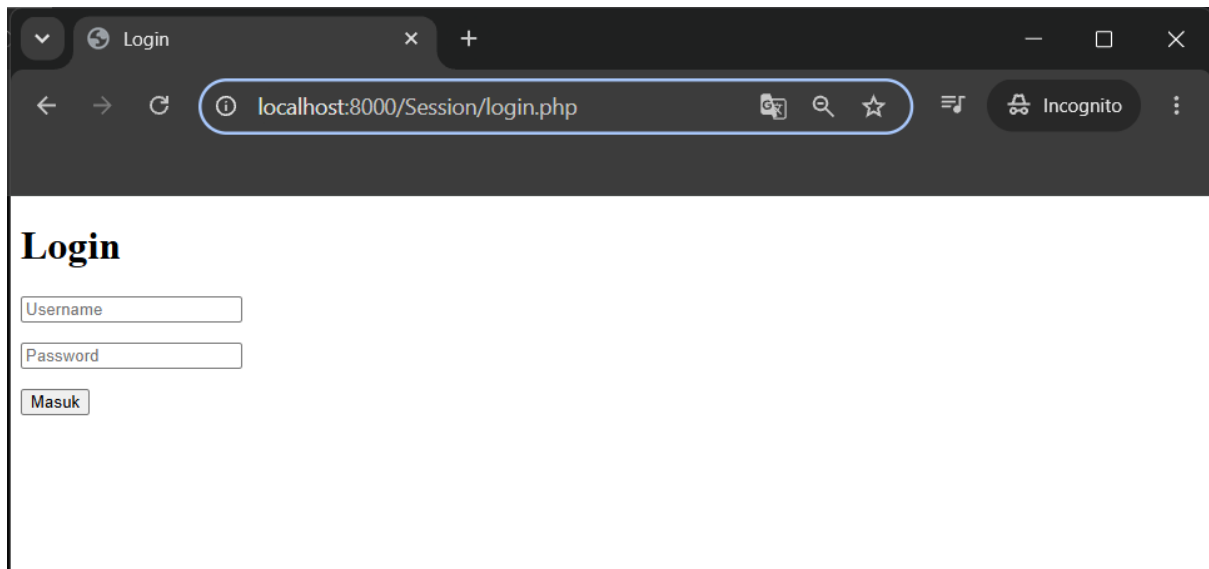


Gambar 16. Login berhasil dan dashboard tampil

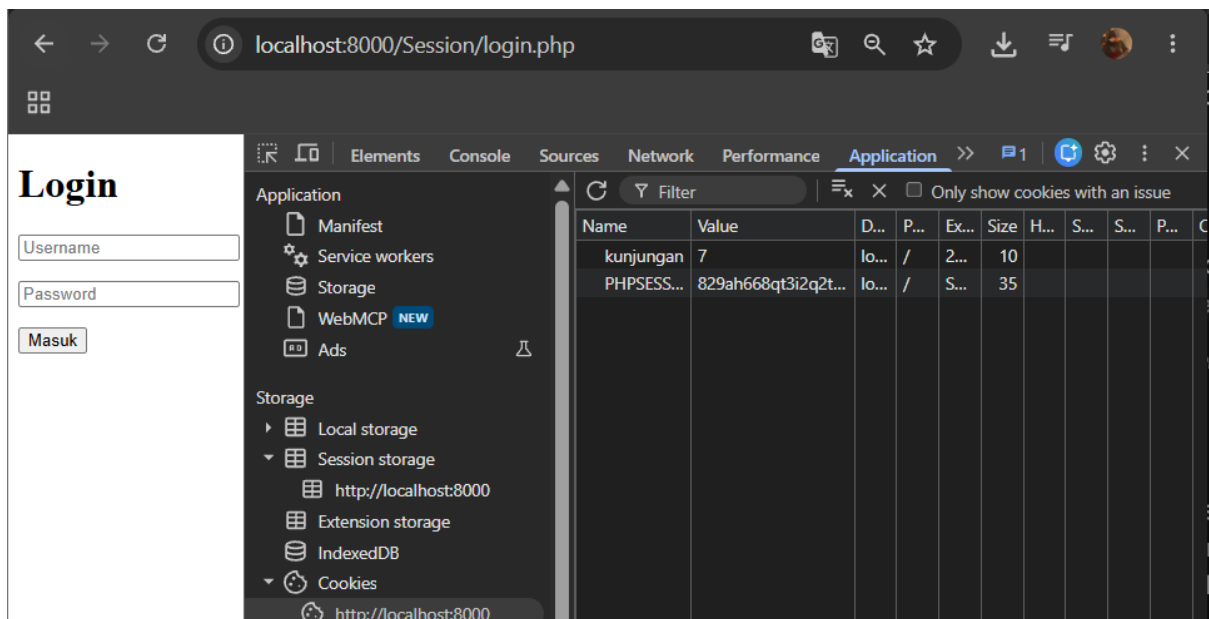


Gambar 17. Browser hanya menyimpan cookie PHPSESSID

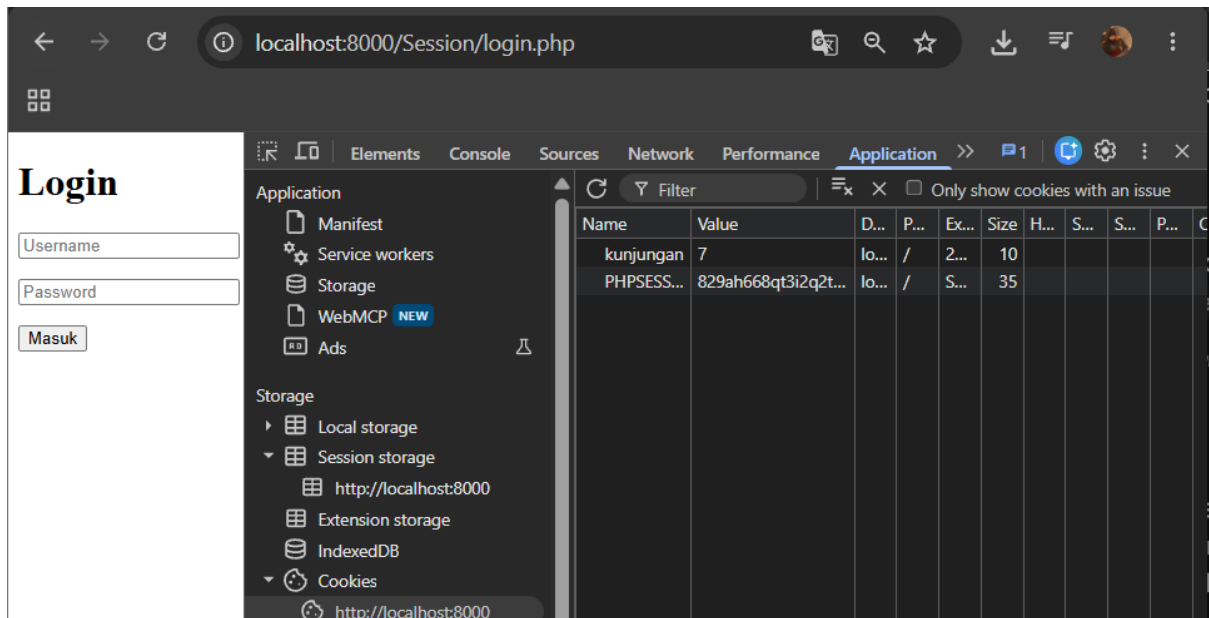
Pada DevTools terlihat bahwa yang tersimpan di browser hanya cookie PHPSESSID berisi ID acak. Nama dan data pengguna tidak disimpan di browser, melainkan di server.



Gambar 18. Membuka dashboard tanpa login (incognito)



Gambar 19. Setelah logout



Gambar 20. Cookie PHPSESSID setelah logout

Dashboard yang dibuka tanpa login langsung dialihkan ke halaman login karena server tidak menemukan `user_id` di session. Setelah logout, data sesi dikosongkan, cookie `PHPSESSID` dihapus, dan sesi dihancurkan di server. Menghapus `PHPSESSID` secara manual lalu me-refresh dashboard juga membuat pengguna dikembalikan ke halaman login karena server tidak lagi mengenali sesinya.

Diskusi: mengapa status login disimpan di session dan bukan di cookie biasa?

Cookie biasa dapat diubah pengguna, sedangkan data session berada di server sehingga tidak bisa dimanipulasi langsung. Seseorang tidak dapat berpura-pura sudah login hanya dengan mengubah isi cookie.

A.5 Tahap 4: Local Storage (JavaScript)

Eksperimen 0: metode dasar di Console

Metode `setItem`, `getItem`, `removeItem`, dan `clear` dicoba langsung di Console DevTools. Dua hal yang ditemukan: nilai angka yang disimpan ternyata dibaca kembali sebagai string, dan array yang disimpan tanpa `JSON.stringify()` rusak menjadi `"[object Object]"`. Karena itu objek dan array harus diubah dengan `JSON.stringify()` saat disimpan dan `JSON.parse()` saat dibaca.

```
> localStorage.setItem('nama', 'Budi');
console.log(localStorage.getItem('nama'));
console.log(localStorage.getItem('tidakada'));

localStorage.setItem('umur', 20);
console.log(typeof localStorage.getItem('umur'));

const mhs = { nama : 'Budi', nim: '12345' };
localStorage.setItem('mhs', JSON.stringify(mhs));

const hasil = JSON.parse(localStorage.getItem('mhs'));
console.log(hasil.nama);

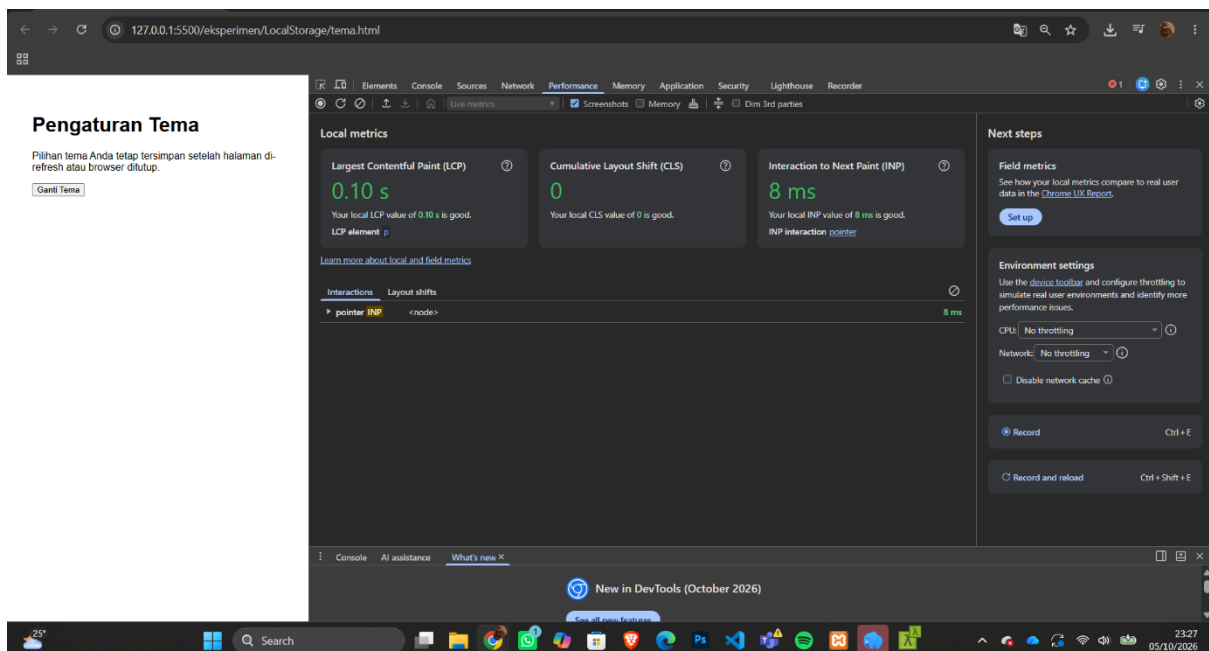
localStorage.removeItem('nama');
localStorage.clear();

Budi VM1110:2
null VM1110:3
string VM1110:6
Budi VM1110:12
< undefined
>
```

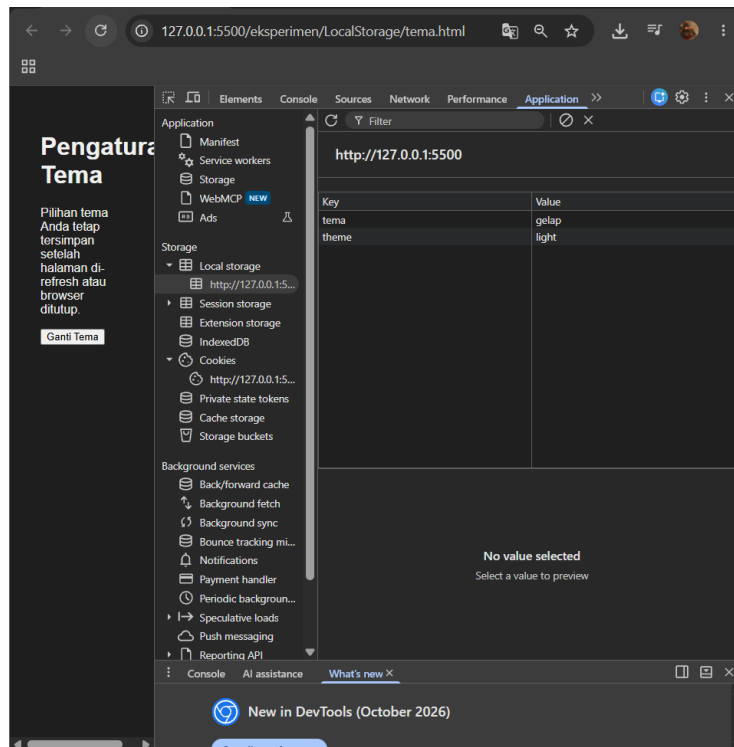
Gambar 21. Mencoba metode Local Storage di Console

Eksperimen 1: tema gelap/terang (tema.html)

Pilihan tema disimpan di Local Storage dengan kunci tema. Setelah tombol Ganti Tema ditekan, tampilan menjadi gelap, dan setelah halaman di-refresh tema gelap tetap dipakai karena dibaca kembali dari Local Storage.



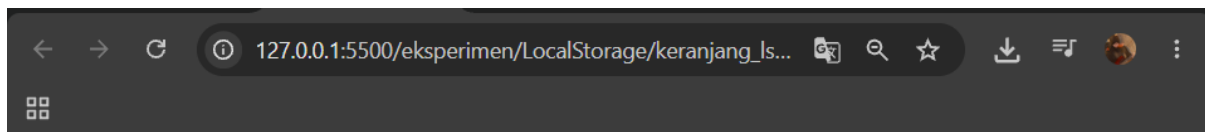
Gambar 22. Tema terang (nilai bawaan)



Gambar 23. Tema gelap tetap dipakai setelah refresh dan Isi Local Storage

Eksperimen 2: keranjang belanja (keranjang_ls.html)

Kasusnya sama dengan keranjang cookie. Hasilnya sama, yaitu Buku Tulis x 2 dan Pulpen x 1 dengan total Rp 13.000. Perbedaannya, data keranjang di Local Storage tidak ikut dikirim ke server pada setiap request.



Daftar Produk

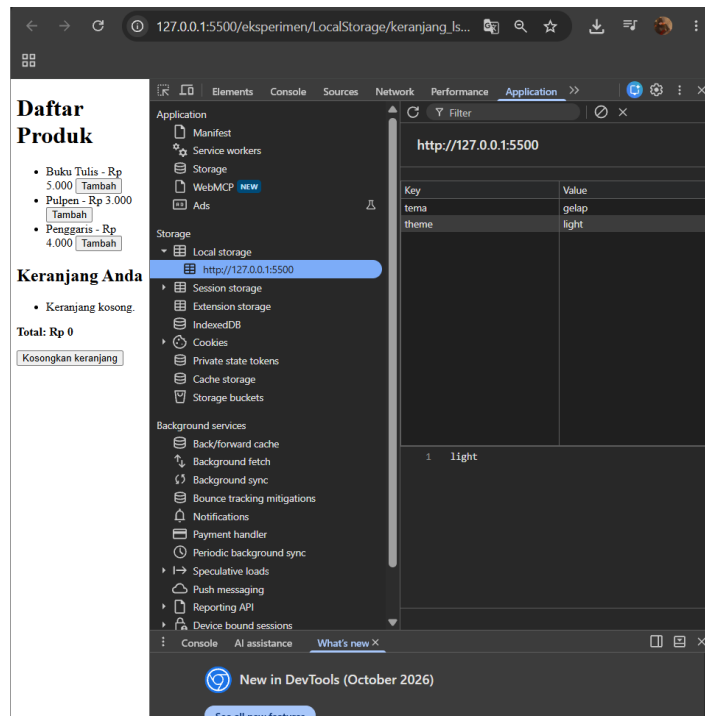
- Buku Tulis - Rp 5.000
- Pulpen - Rp 3.000
- Penggaris - Rp 4.000

Keranjang Anda

- Keranjang kosong.

Total: Rp 0

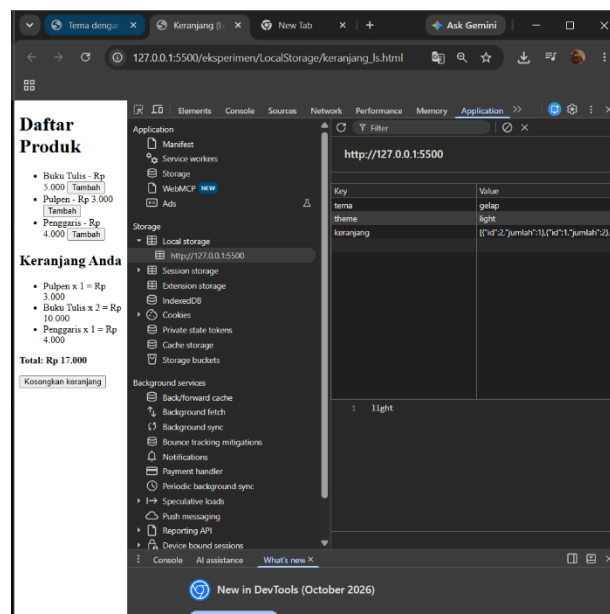
Gambar 25. Keranjang dengan Local Storage



Gambar 26. Kunci keranjang di Local Storage

Eksperimen 3: localStorage vs sessionStorage (storage_beda.html)

Kedua storage memiliki API yang sama. Setelah tab ditutup lalu halaman dibuka lagi di tab baru, data dari localStorage masih ada, sedangkan data dari sessionStorage sudah hilang.



Gambar 27. Perbandingan localStorage dan sessionStorage

Pertanyaan: apakah mengubah keranjang di Local Storage lebih aman dibanding cookie?

Tidak. Keduanya dapat diubah pengguna lewat DevTools. Bedanya, Local Storage tidak ikut terkirim ke server pada setiap request.

A.6 Tahap 5: Perbandingan dan Keamanan

Aspek	Cookies	Session (PHP)	Local Storage	sessionStorage
Lokasi data	Browser	Server (ID di browser)	Browser	Browser
Kapasitas	~4 KB	Bergantung server	~5 MB	~5 MB
Masa berlaku	Diatur lewat expires	Sampai sesi berakhir/timeout	Permanen sampai dihapus	Sampai tab ditutup
Dikirim ke server	Ya, setiap request	Hanya ID sesi	Tidak	Tidak
Diakses oleh	PHP dan JavaScript (kecuali HttpOnly)	PHP	JavaScript	JavaScript
Cocok untuk	Preferensi kecil yang dibaca server	Status login, data sensitif	Data besar atau preferensi sisi klien	Data sementara per tab

Tabel 1. Perbandingan mekanisme penyimpanan data

Simulasi risiko XSS pada Local Storage

Local Storage diisi data lalu dibaca dengan perintah `console.log(JSON.stringify(localStorage));`. Seluruh isi Local Storage langsung tampil dalam satu string JSON, yaitu `{"token_rahasia":"xyz123abc"}`. Artinya, skrip jahat yang berhasil disisipkan ke halaman (XSS) dapat membaca semua data di Local Storage hanya dengan satu baris kode.

9

```
> console.log(JSON.stringify(localStorage));
{}
VM1280:1
< undefined
> localStorage.setItem('token_rahasia', 'xyz123abc');
console.log(JSON.stringify(localStorage));
{"token_rahasia":"xyz123abc"}
VM1519:2
< undefined
>
```

Gambar 28. Skrip apa pun di halaman dapat membaca seluruh isi Local Storage

Kesimpulan keamanan

- Password, token akses, dan data pribadi sensitif tidak boleh disimpan di Local Storage maupun cookie biasa.
- Output dari pengguna harus selalu di-escape, yaitu dengan `htmlspecialchars()` di PHP dan `textContent` (bukan `innerHTML`) di JavaScript.

- Cookie yang tidak perlu dibaca JavaScript sebaiknya memakai flag `HttpOnly`, ditambah `Secure` dan `SameSite`.
- Status login dan data sensitif disimpan di session; keranjang pengguna yang sudah login sebaiknya di database agar konsisten antar perangkat.

B. Hasil Tugas 1 : Login dengan Password Terenkripsi

Bagian ini memuat hasil Tugas 1, yaitu sistem login dengan Laravel 13 yang memakai password ter-hash, proteksi halaman lewat middleware, dan logout yang aman. Logika login ditulis sendiri tanpa starter kit (Breeze, Jetstream, dan sejenisnya).

B1. Implementasi

Proyek dibuat dengan composer create-project laravel/laravel:^13.0 tugas1 dan memakai MySQL dengan database db_tugas. Komponen yang dibuat:

- Migration: tabel users memakai kolom id, username (unik), password, nama_lengkap, dan timestamps. Kolom email bawaan diganti menjadi username.
- Model User: \$fillable disesuaikan menjadi username, password, nama_lengkap, dan memakai cast 'password' menjadi 'hashed'.
- Seeder dibuat menggunakan file DatabaseSeeder.php dengan mendaftarkan 2 akun user: budi (Budi Santoso) dan siti (Siti Aminah) dengan password ter-hash menggunakan Hash::make('rahasia123!').
- AuthController: memuat method showLogin, login, dashboard, dan logout.
- View Blade: login.blade.php dan dashboard.blade.php.

Daftar Route

Method	URL	Fungsi	Middleware
GET	/login	Menampilkan form login (nama route login)	guest
POST	/login	Validasi input dan Auth::attempt()	guest
GET	/dashboard	Menampilkan "Selamat datang, [nama_lengkap]!"	auth
POST	/logout	Mengakhiri sesi	auth

Alur proses login:

1. Input username dan password divalidasi (wajib diisi).
2. Auth::attempt() membandingkan password input dengan hash di database.
3. Jika berhasil, \$request->session()->regenerate() dipanggil dan pengguna diarahkan ke /dashboard.

4. Jika gagal, pengguna dikembalikan ke form dengan pesan “Username atau password salah.” tanpa menyebut mana yang salah, dan username tetap terisi.

B.2 Hasil Pengujian

phpMyAdmin

Recent

Favorites

db_toko_online

New

cache

cache_locks

failed_jobs

jobs

job_batches

migrations

orders

order_details

password_reset_tokens

products

sessions

users

db_tugas

New

cache

cache_locks

failed_jobs

Server: 127.0.0.1 > Database: db_tugas > Table: users

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

Operations

Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

SELECT * FROM `users`

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

id

username

password

nama_lengkap

created_at

☐

Edit

Copy

Delete

1

budi

\$2y\$12\$hebz0hCrEiTYbfH/g1NOQui9M3NONn1DEQmfvYywnT3...

Budi Santoso

2026-10-06 17:10:09

☐

Edit

Copy

Delete

2

siti

\$2y\$12\$WuU6amqlFdreLhLfrO7W7e6zihbcxv/mia.5xpCuNxu...

Siti Aminah

2026-10-06 17:10:09

Check all

With selected:

Edit

Copy

Delete

Export

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Query results operations

Print

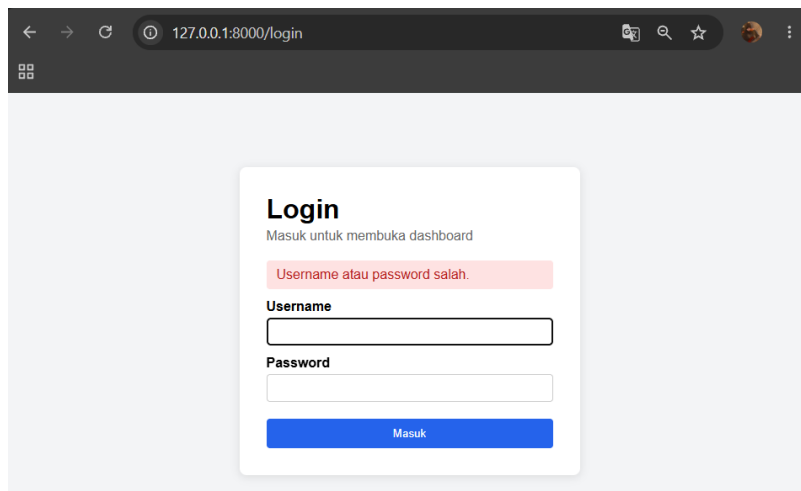
Copy to clipboard

Export

Display chart

Create view

Gambar 29. Isi tabel *users*: kolom password berupa hash



127.0.0.1:8000/login

Login

Masuk untuk membuka dashboard

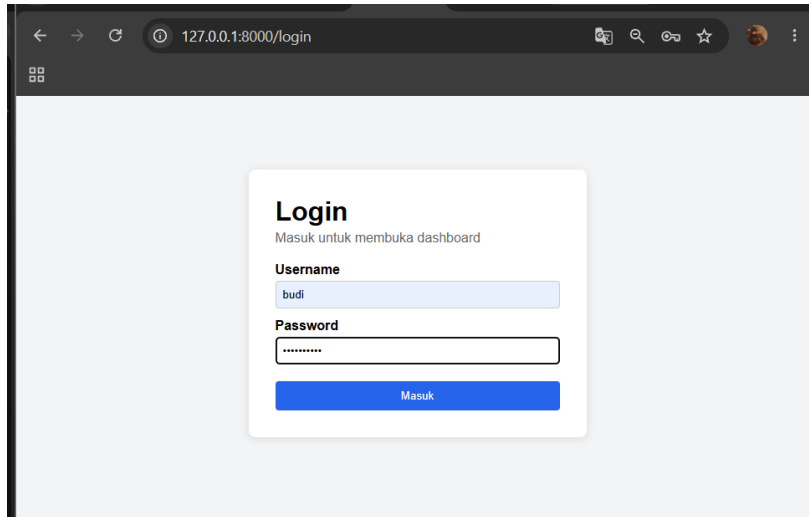
Username atau password salah.

Username

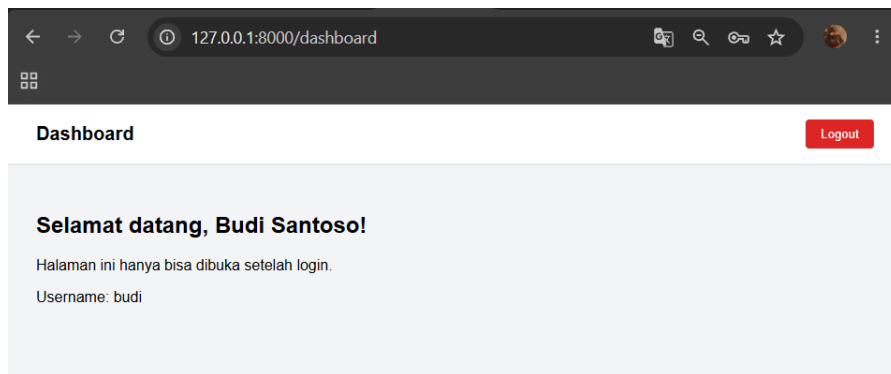
Password

Masuk

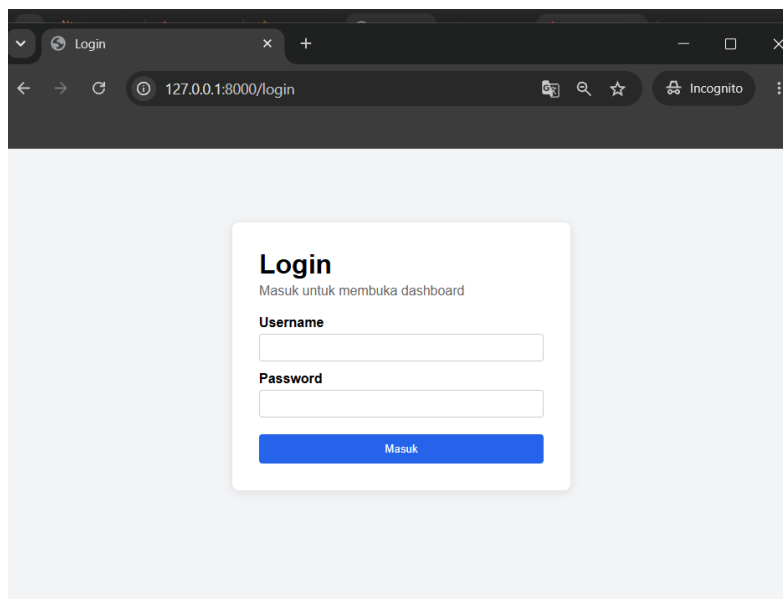
Gambar 30. Login gagal: pesan “Username atau password salah.”



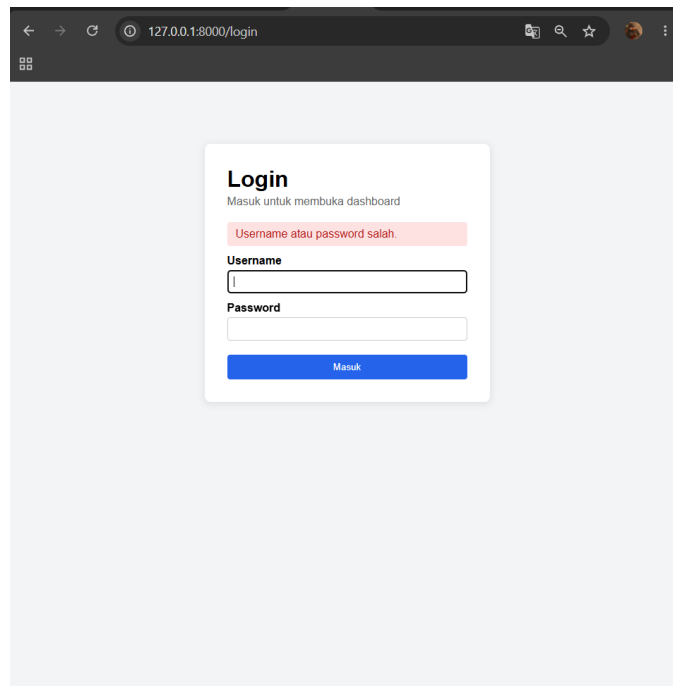
Gambar 31. Login berhasil: form terisi sebelum tombol Masuk ditekan



Gambar 32. Halaman dashboard setelah login berhasil



Gambar 33. Dashboard dibuka di jendela incognito tanpa login: dialihkan ke /login



Gambar 34. Halaman login setelah logout

Pada pengujian pertama, kolom password di tabel users terisi hash, bukan teks asli. Login dengan password salah menampilkan pesan umum tanpa membocorkan apakah username atau password yang keliru. Login dengan data benar mengarahkan pengguna ke dashboard yang menyapa dengan nama_lengkap. Saat /dashboard dibuka langsung lewat URL di jendela incognito, pengguna otomatis dialihkan ke /login karena middleware auth tidak menemukan sesi yang valid. Setelah logout, pengguna kembali ke halaman login dan tidak bisa membuka dashboard lagi tanpa login ulang.

B.3 Jawaban Pertanyaan

a. Mengapa password harus di-hash? Lihat kolom password di tabel users dan jelaskan isinya.

Hash adalah fungsi satu arah: password diubah menjadi deretan karakter yang tidak bisa dikembalikan ke aslinya. Jika database bocor atau dibaca orang yang tidak berhak (termasuk admin database), penyerang hanya mendapat hash, bukan password asli yang sering dipakai ulang di layanan lain. Kolom password berisi string dengan panjang 60 karakter. Itu hash bcrypt: \$2y\$ menunjukkan algoritma bcrypt, angka berikutnya adalah cost factor, lalu diikuti salt acak dan hash-nya. Karena ada salt, dua user dengan password yang sama menghasilkan hash yang berbeda. Saat login, Auth::attempt() membandingkan password input dengan hash memakai Hash::check(), bukan membandingkan teks asli.

b. Mengapa logout memakai metode POST dengan @csrf, bukan link biasa?

Link biasa memakai GET, padahal GET seharusnya hanya untuk membaca data dan tidak mengubah state. Jika logout memakai link, siapa pun bisa memaksa

pengguna logout, misalnya lewat tag `` di halaman lain, atau browser bisa memicunya otomatis lewat prefetch. Dengan POST dan `@csrf`, request hanya diterima jika membawa token CSRF yang valid dan terikat pada sesi pengguna, sehingga situs lain tidak bisa memalsukan request tersebut (serangan CSRF).

c. Apa fungsi middleware auth dan guest? Apa yang terjadi jika /dashboard dibuka tanpa login?

Middleware adalah lapisan pemeriksa yang berjalan sebelum request sampai ke controller. Middleware auth hanya mengizinkan pengguna yang sudah login; yang belum login dialihkan ke route bernama login. Middleware guest kebalikannya: hanya untuk pengguna yang belum login, sehingga yang sudah login diarahkan ke dashboard dan tidak perlu melihat form login lagi. Jika /dashboard dibuka tanpa login, middleware auth mencegat request dan mengalihkan pengguna ke /login, sehingga controller dashboard tidak pernah dijalankan.

d. Apa fungsi session()->regenerate() setelah login berhasil?

Fungsi ini membuat ID sesi yang baru setelah login berhasil, sambil mempertahankan data sesinya. Tujuannya mencegah serangan session fixation: penyerang yang sebelumnya memaksa korban memakai ID sesi tertentu tidak bisa memakai ID itu lagi karena sudah diganti begitu korban login. Ini konsep yang sama dengan `session_regenerate_id(true)` pada eksperimen session PHP di Bagian A.4.

C. Hasil Tugas 2: Keranjang Belanja Tanpa Login (Laravel)

Tugas 2 membuat aplikasi keranjang belanja sederhana tanpa login. Source code berada di folder tugas-2-keranjang pada repository GitHub yang dicantumkan di halaman judul. Mekanisme penyimpanan yang dipilih adalah **session**.

C.1 Implementasi dan Keputusan Teknis

Data produk (**[jumlah]** produk dengan kolom id, nama, harga, stok) disimpan di tabel barang pada MySQL dan diisi lewat seeder. Keranjang disimpan di session dengan kunci cart berbentuk array `[id_produk => jumlah]`. Sesuai ketentuan, yang disimpan hanya id produk dan jumlah. Nama dan harga selalu diambil dari tabel barang saat halaman ditampilkan.

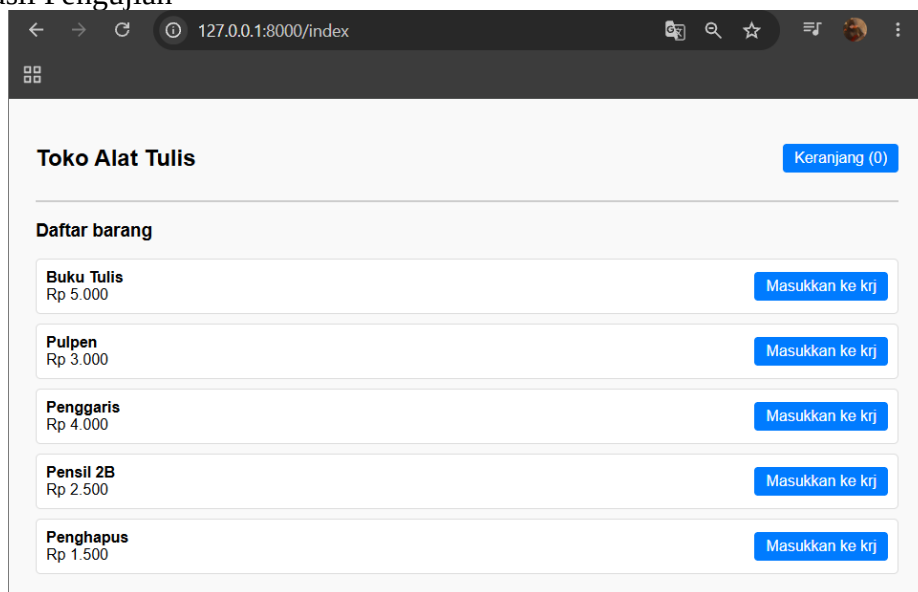
Fitur yang dibuat:

Fitur	Route	Perilaku
Daftar barang + jumlah item	GET /	Menampilkan semua produk dan jumlah item di header
Tambah ke keranjang	POST /keranjang/tambah/{id}	Jika produk sudah ada, jumlah bertambah
Lihat keranjang	GET /keranjang	Menampilkan harga, jumlah, subtotal, dan total dalam format rupiah

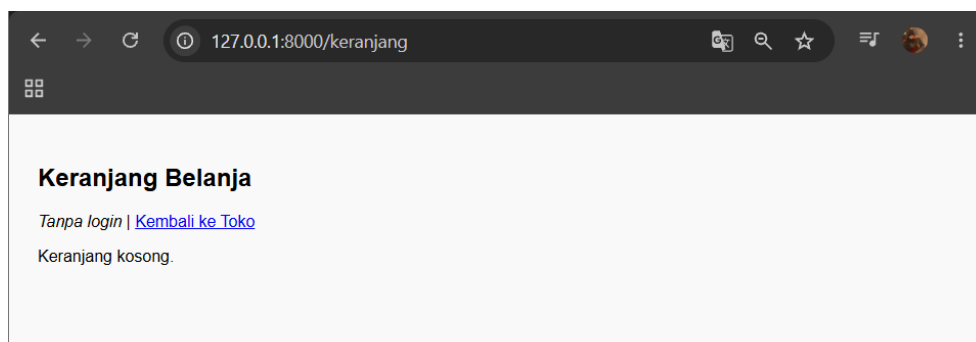
Tombol + dan -	[route]	Mengubah jumlah; jika menjadi 0, item dihapus otomatis
Hapus satu item	[route]	Menghapus satu produk dari keranjang
Kosongkan keranjang	[route]	Menghapus seluruh isi keranjang

Session dipilih karena data keranjang tinggal di server, sedangkan browser hanya menyimpan ID sesi dalam cookie. Pengguna tidak dapat melihat atau mengubah isi keranjang secara langsung, dan data tidak ikut terkirim bolak-balik pada setiap request. Pilihan ini juga paling mudah dikembangkan menjadi proses checkout, karena server sudah memegang data keranjang dan bisa memvalidasinya terhadap tabel barang.

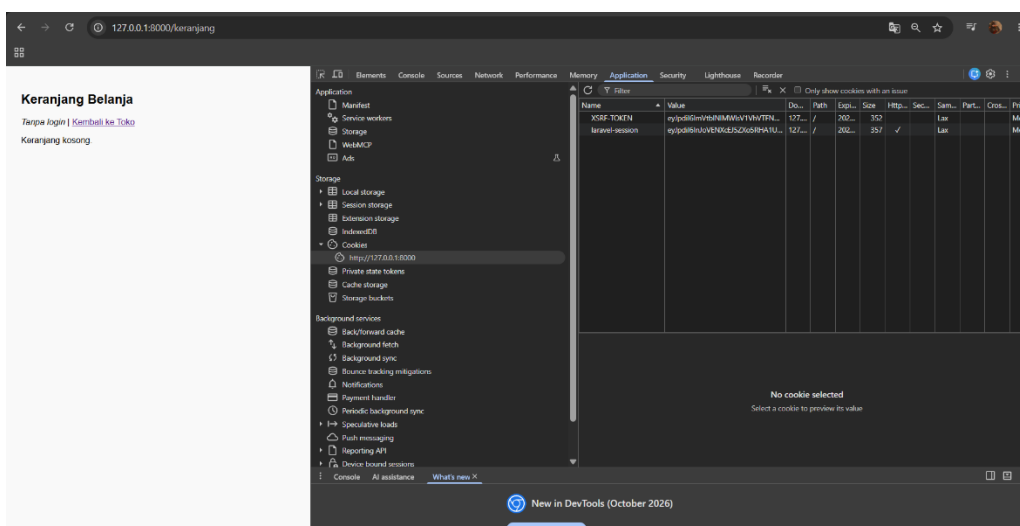
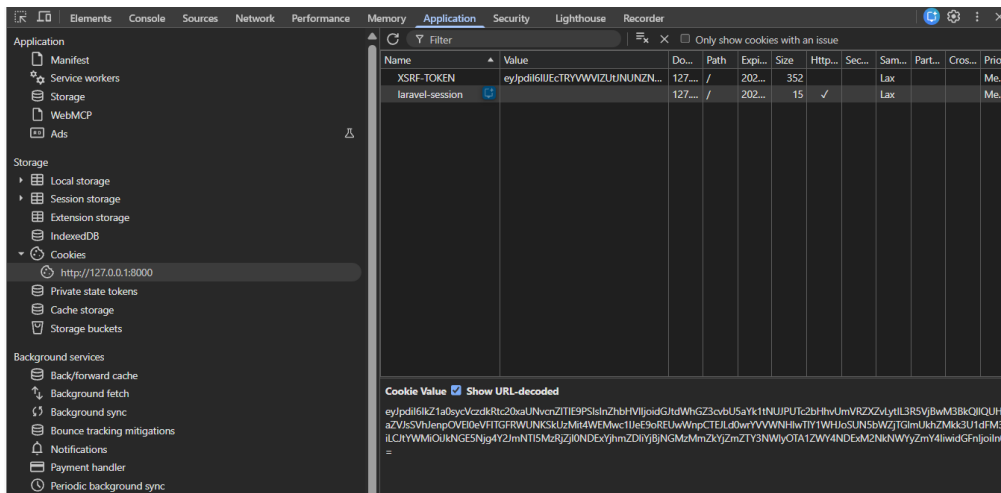
C.2 Hasil Pengujian



Gambar 35. Halaman daftar barang



Gambar 36. Keranjang kosong



Berdasarkan pengujian ketahanan data (data persistence), saat halaman di-refresh, data keranjang tetap tersimpan dengan utuh karena ID sesi dibawa kembali via cookie laravel_session. Ketika tab browser ditutup lalu dibuka kembali pada URL yang sama, isi keranjang tetap ada karena masa berlaku cookie session Laravel secara bawaan diatur selama 120 menit (SESSION_LIFETIME=120). Namun, ketika seluruh browser ditutup (close browser) atau dibuka melalui jendela Incognito baru, keranjang kembali kosong karena browser membuat ID sesi baru yang berbeda dari sesi sebelumnya di server.

C.3 Jawaban Pertanyaan

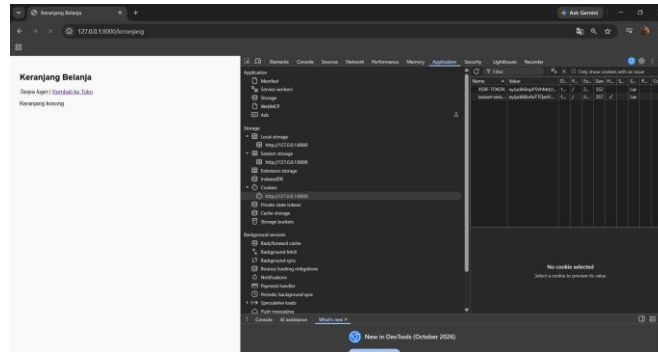
1. Mekanisme apa yang Anda pilih, dan mengapa?

Saya memilih session. Alasannya, data keranjang disimpan di server dan browser hanya memegang ID sesi, sehingga pengguna tidak dapat mengubah jumlah barang secara langsung. Karena aplikasi sudah memakai Laravel dan produk berada di MySQL, session mudah dipakai dan tidak memerlukan kode JavaScript tambahan untuk menyimpan data. Session juga memudahkan pengembangan ke checkout, karena server sudah memiliki data keranjang yang bisa divalidasi.

2. Satu kelebihan dan satu kekurangan dibanding dua mekanisme lainnya.

- Kelebihan: data berada di server sehingga tidak dapat diubah pengguna lewat DevTools (berbeda dengan cookies dan Local Storage yang isinya bisa diedit), dan datanya tidak ikut dikirim pada setiap request seperti cookies.
- Kekurangan: memakai sumber daya server (penyimpanan session untuk setiap pengunjung) dan datanya hilang setelah sesi berakhir. Local Storage tidak membebani server dan tetap tersimpan sampai dihapus.

3. Mengubah data keranjang lewat DevTools (sertakan screenshot).



Isi keranjang tidak dapat diubah lewat DevTools karena tidak berada di browser. Cookie yang terlihat di Application > Cookies hanya berisi ID sesi (nilainya terenkripsi dan acak), sedangkan jumlah barang tersimpan di server. Saya mencoba mengubah nilai cookie sesi. Hasilnya server tidak mengenali sesi tersebut, membuat sesi baru, dan keranjang tampil kosong. Jadi mengubah cookie tidak membuat jumlah menjadi 999; paling jauh hanya membuat pengguna kehilangan keranjangnya. Ini berbeda dengan percobaan keranjang cookie di Bagian A.3, yang total harganya berubah menjadi Rp 4.998.000.

4. Mengapa harga tidak boleh dipercaya dari data yang dikirim atau disimpan di browser? Apa yang harus dilakukan server saat checkout?

Data di browser dapat dilihat dan diubah siapa saja lewat DevTools atau dengan memodifikasi request, sehingga pengguna bisa mengubah harga menjadi Rp 1 atau jumlah menjadi tidak wajar. Karena itu server tidak boleh memercayai harga dari browser. Saat checkout, server harus: (1) mengambil harga terbaru dari tabel barang berdasarkan id produk, (2) memeriksa bahwa produk masih ada dan stoknya cukup, (3) menghitung ulang subtotal dan total sendiri, dan (4) baru membuat pesanan dari hasil perhitungan itu.

5. Apa yang terjadi pada keranjang jika pengguna membuka situs dari browser lain?

Keranjang tampil kosong. Session terikat pada cookie ID sesi yang tersimpan di browser tertentu, sedangkan browser lain tidak memiliki cookie itu sehingga server memberinya sesi baru yang kosong. Keranjang yang sebelumnya diisi tetap ada di server, tetapi hanya dapat diakses dari browser yang membawa ID sesi yang sama. Untuk keranjang yang konsisten antar browser atau perangkat, pengguna harus login dan keranjang disimpan di database, seperti yang disimpulkan di Bagian A.6.

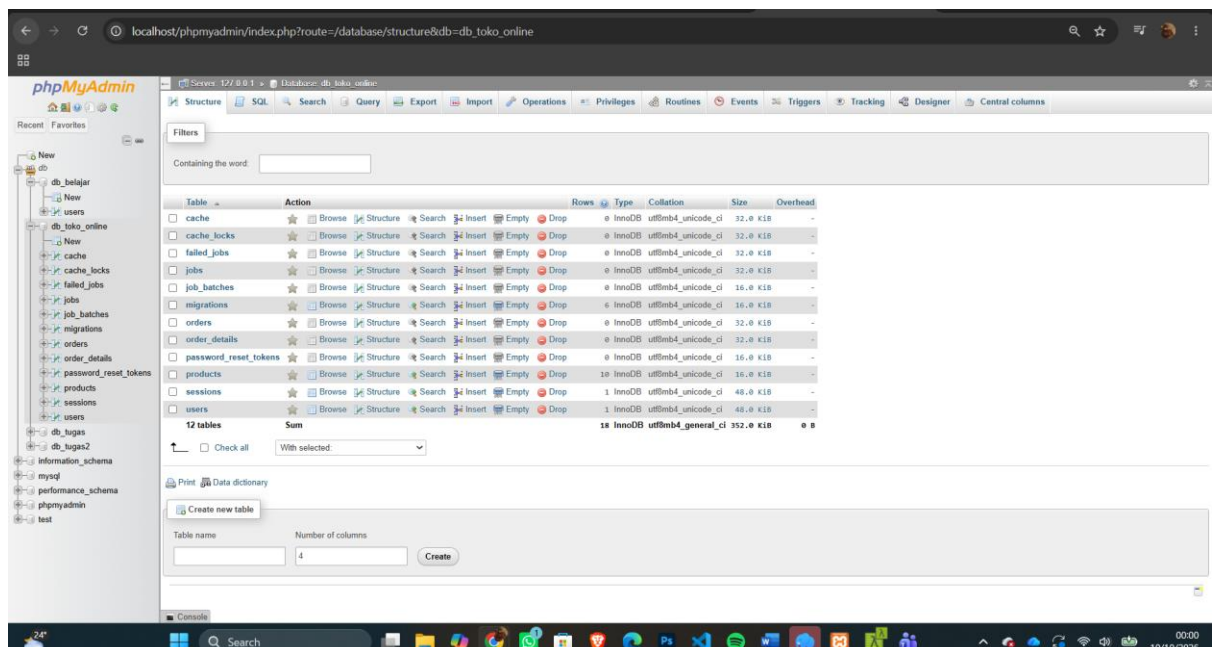
D. Hasil Tugas Toko Online

Tugas ini membuat program toko online berbasis web dengan Laravel dan MySQL. Source code berada di folder tugas-toko-online pada repository GitHub yang dicantumkan di halaman judul. Pengunjung boleh melihat daftar barang tanpa login, tetapi untuk memasukkan barang ke keranjang dan checkout pengguna wajib login.

D.1 Rancangan Database

Database terdiri dari empat tabel sesuai rancangan pada soal:

Tabel	Kolom Utama	Keterangan
users	id_user (PK), nama_lengkap, email (unik), username (unik), password (hash), no_hp, alamat	Data pengguna/pembeli
products	id_barang (PK), nama_barang, deskripsi, harga, stok, gambar	Minimal 10 barang alat tulis
orders	id_order (PK), id_user (FK), tanggal_order, total_harga, alamat_pengiriman	Satu baris per transaksi checkout
order_details	id_order + id_barang (PK gabungan), harga_satuan, jumlah_beli	Satu baris per item dalam pesanan



Gambar 42. Struktur tabel di PhpMyAdmin

Kolom harga_satuan pada order_details menyimpan harga saat barang dibeli sebagai arsip, sehingga riwayat pesanan tidak berubah walaupun harga di tabel products diubah di kemudian hari.

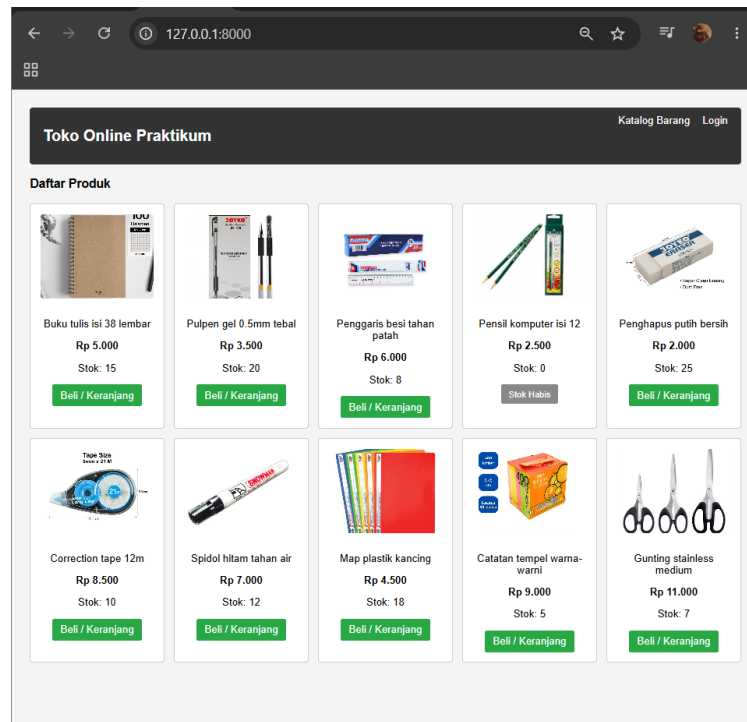
D.2 Implementasi Fitur

1. Login & Autentikasi: Pengguna login menggunakan username (misal: budi) dan password (di-hash menggunakan Bcrypt). Halaman katalog daftar barang bisa dibuka siapa saja secara publik, sedangkan route keranjang belanja, checkout, dan riwayat pesanan dilindungi oleh middleware auth. Jika pengguna belum login mencoba mengeklik "Beli / Keranjang", sistem secara otomatis mengarahkannya ke halaman login.
2. Daftar Barang (Katalog): Setiap barang menampilkan foto produk dari folder public/images/ (format .png), nama barang, deskripsi, harga, dan stok. Barang dengan stok 0 (seperti Pensil 2B Castelli) ditandai dengan tombol "Stok Habis" dan dinonaktifkan dari pembelian.
3. Keranjang Belanja: Pengguna yang sudah login dapat menambah barang, mengubah jumlah barang, menghapus barang, dan melihat subtotal serta total belanjaan. Jumlah barang yang dibeli dibatasi tidak boleh melebihi stok yang tersedia di database, di mana pemeriksaannya dilakukan secara ketat di sisi server (Server-Side Validation). Keranjang disimpan menggunakan Laravel Session.
4. Checkout: Proses checkout dijalankan di sisi server dengan urutan:
 - Mengambil data barang dan harga resmi terbaru langsung dari tabel products, bukan dari data keranjang di browser.
 - Memeriksa bahwa stok semua barang yang ada di keranjang masih mencukupi.
 - Membuat satu record baru di tabel orders dengan id_order unik berbasis timestamp dan total_harga yang dihitung oleh server.
 - Menyimpan setiap item barang ke tabel order_details beserta harga_satuan saat transaksi berlangsung.
 - Mengurangi stok barang pada tabel products secara otomatis menggunakan metode decrement().
 - Mengosongkan keranjang belanja session (session()->forget('cart')).

Jika salah satu langkah gagal, seluruh proses dibatalkan sehingga tidak ada pesanan setengah jadi atau stok yang berkurang tanpa pesanan. Ongkos kirim diabaikan, sehingga total bayar sama dengan total harga barang.

5. Riwayat Pesanan: Pengguna dapat melihat daftar transaksi miliknya (id_order, tanggal, total bayar, alamat pengiriman) beserta rincian item barang yang dibeli. Query dibatasi khusus pada id_user pengguna yang sedang aktif login, sehingga pengguna tidak bisa mengintip pesanan orang lain.

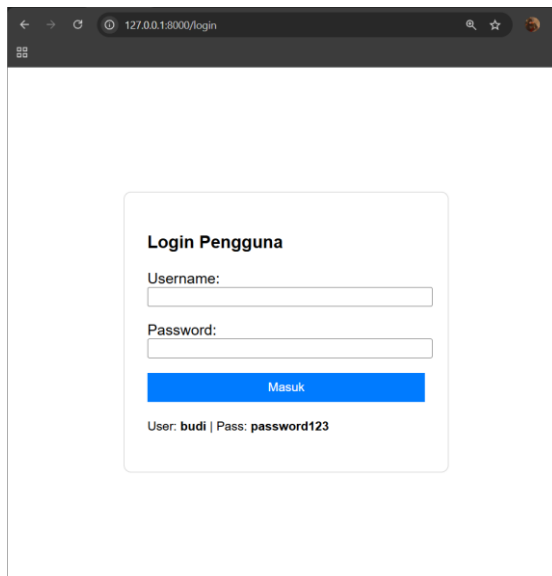
D.3 Hasil Pengujian



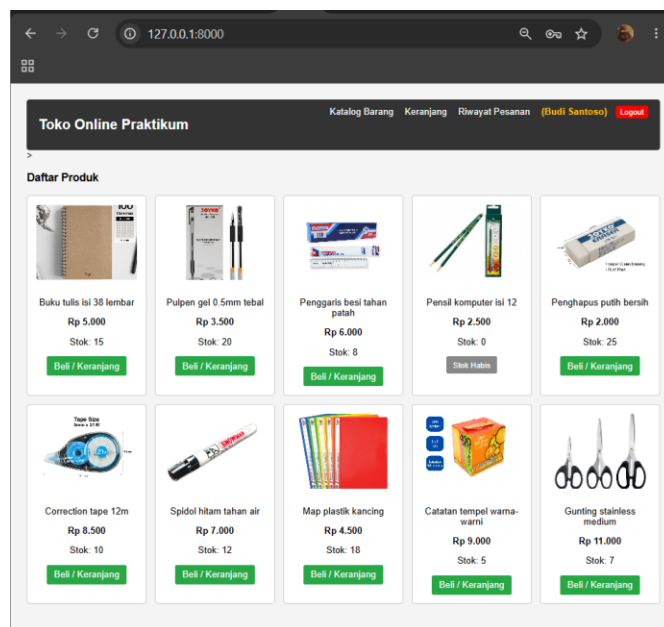
Gambar 43. Halaman daftar barang katalog produk tanpa login (menampilkan 10 item alat tulis)



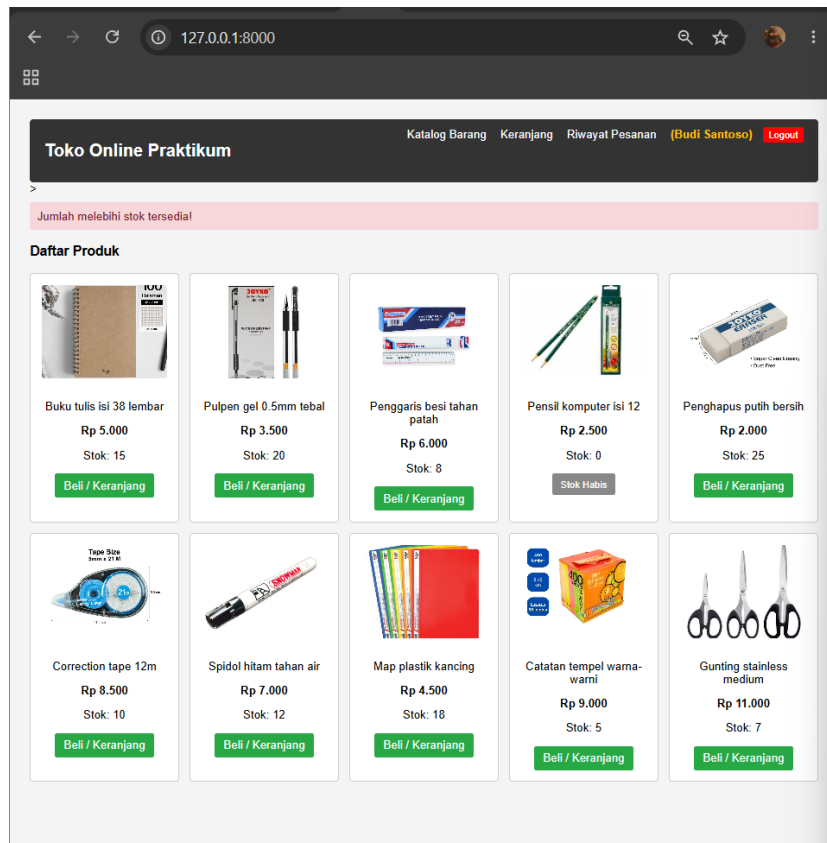
Gambar 44. Barang dengan stok 0 ("Pensil 2B Castelli") tombol belinya dinonaktifkan



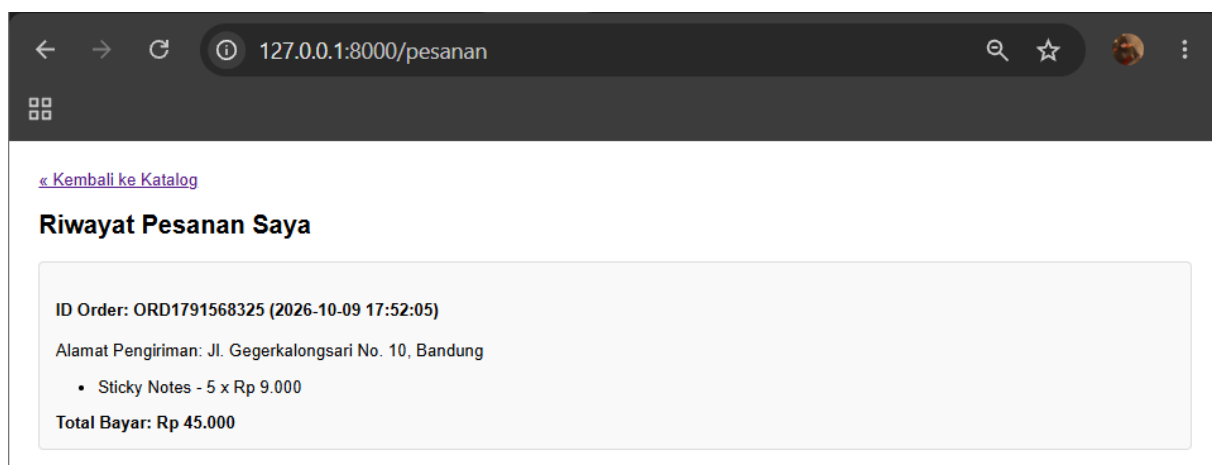
Gambar 45. Pengalihan otomatis ke halaman login saat pengunjung belum terautentikasi menekan tombol beli



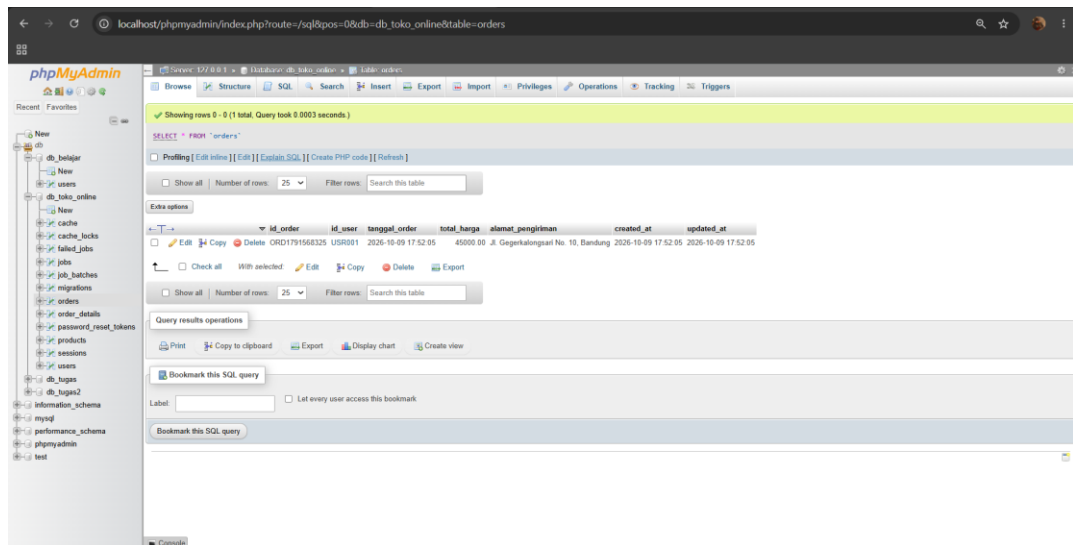
Gambar 46. Halaman keranjang belanja terisi setelah pengguna (budi) berhasil login



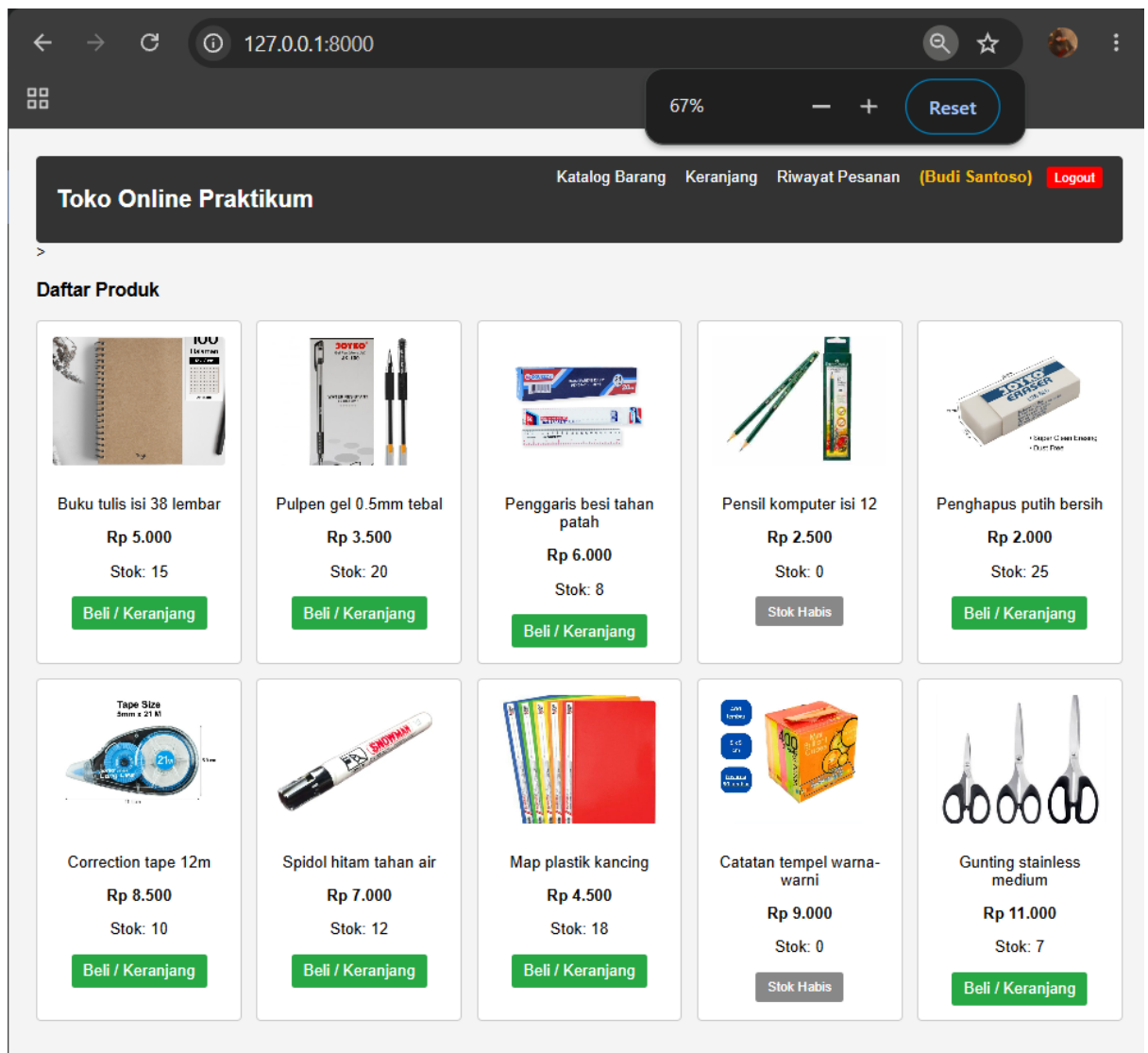
Gambar 47. Notifikasi/peringatan sistem saat memasukkan jumlah barang melebihi stok



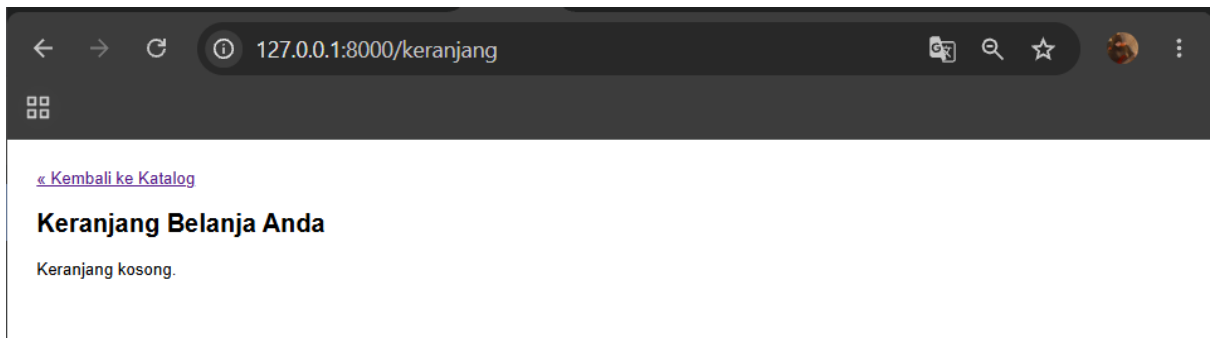
Gambar 48. Halaman checkout dan notifikasi pesanan berhasil dibuat



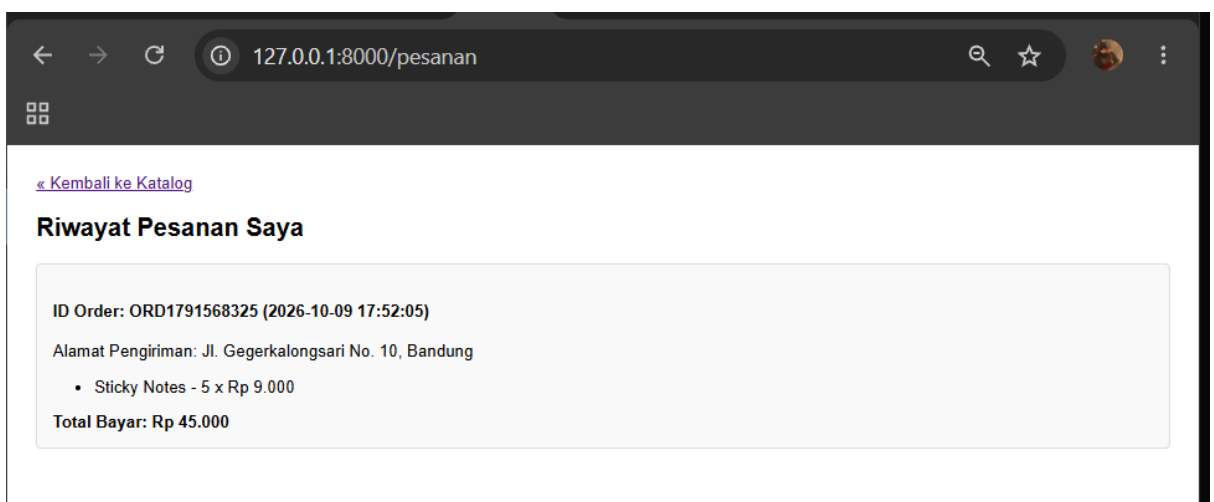
Gambar 49. Record data baru pada tabel orders dan order_details di database db_toko_online



Gambar 50. Pengurangan jumlah stok produk pada tabel products sebelum dan sesudah checkout (contoh: stok Buku Tulis berkurang dari 15 menjadi 13)



Gambar 51. Halaman keranjang belanja kembali kosong setelah checkout selesai



Gambar 52. Halaman riwayat pesanan yang menampilkan daftar transaksi dan detail item milik pengguna budi

Penjelasan Pengujian: Pengujian menunjukkan bahwa pengunjung dapat melihat barang tanpa login, tetapi aksi pembelian langsung mengarahkan pengguna ke halaman login. Pada proses pengujian transaksi checkout oleh user budi, produk yang dibeli adalah Catatan tempel warna-warni (Sticky Notes) sebanyak 5 unit dengan harga satuan Rp 9.000 (Total Rp 45.000). Sebelum checkout dilakukan, stok Catatan tempel pada katalog adalah 5 unit (Gambar 46). Setelah proses checkout berhasil, transaksi tercatat di riwayat pesanan dengan ID Order ORD1791568325. Pada tabel products di database, stok Catatan tempel warna-warni otomatis berkurang 5 unit menggunakan metode decrement(), sehingga stoknya di katalog produk berubah dari 5 menjadi 0 (Stok Habis) (Gambar 50).

E. Penjelasan Keputusan Teknis

1. Session untuk Keranjang Belanja: Data keranjang disimpan di server menggunakan Laravel Session sehingga tidak bisa diubah atau dimanipulasi oleh pengguna melalui DevTools browser.

2. Penyimpanan ID dan Quantity: Session keranjang hanya menyimpan id_barang dan jumlah_beli. Nama barang dan harga selalu diambil ulang dari database MySQL agar tidak ada manipulasi harga dari sisi client.
3. Keamanan Autentikasi & Session: Password disimpan dalam bentuk hash menggunakan algoritma Bcrypt. Proses login memanfaatkan Auth::attempt() dengan session()->regenerate() untuk mencegah serangan session fixation, serta logout menggunakan metode POST yang dilindungi CSRF Token.
4. Integritas Data Transaksi Checkout: Pengurangan stok dan pencatatan riwayat transaksi dilakukan di server secara sekuensial dan tervalidasi agar terhindar dari kondisi inconsistent state (stok berkurang tetapi pesanan gagal dibuat).
5. Arsip Harga Satuan: Kolom harga_satuan disimpan secara independen pada tabel order_details sebagai arsip histori untuk menjaga akurasi laporan keuangan transaksi meskipun harga barang pada tabel products mengalami kenaikan atau penurunan di kemudian hari.
6. Server-Side Validation: Seluruh validasi (stok barang, otentikasi login, dan kepemilikan riwayat pesanan) wajib divalidasi di sisi server karena validasi di sisi browser (JavaScript/HTML) sangat mudah dilewati oleh pengguna.

F. Lesson Learned

Pengerjaan Tugas Toko Online menggunakan Laravel 13 dan MySQL memberikan pemahaman mendalam terkait autentikasi serta pengelolaan session pada aplikasi web modern. Masalah utama hilangnya status login saat pengguna menekan tombol keranjang di lingkungan lokal terbukti diselesaikan dengan mengkonfigurasi SESSION_DRIVER=file pada file .env serta mengisolasi pemanggilan Auth::login(\$user) secara eksplisit. Selain itu, penetapan id_user bertipe VARCHAR sebagai Primary Key menuntut penyesuaian fungsionalitas getAuthIdentifierName() pada model User agar Laravel dapat mengidentifikasi kolom identifier pengguna secara tepat.

Di sisi transaksi dan keamanan, penanganan error MethodNotAllowedHttpException (405) menegaskan pentingnya keselarasan metode pengiriman form (POST) dengan definisi route di web.php. Pengalaman ini juga memberikan gambaran utuh alur transaksi e-commerce, mulai dari pembatasan stok di sisi server (server-side validation), pencatatan arsip harga_satuan pada order_details guna menjaga histori harga, hingga pembersihan session keranjang setelah checkout berhasil.